

Deployment and Validation of a Mail Server Threat Detection and Protection Framework

Chapter 1

Introduction

1.1 Background

Email has become one of the most widely used communication methods for both individuals and organizations. Businesses, educational institutions, and government agencies rely heavily on email for exchanging information, sharing documents, and supporting daily operations. As the use of email continues to grow, it has also become one of the most common targets for cyber threats.

Modern email systems are frequently exposed to attacks such as spam campaigns, phishing attempts, malware distribution, email spoofing, and unauthorized SMTP relay. These threats can affect the confidentiality, integrity, and availability of information while also disrupting organizational communication. As a result, simply deploying a mail server is no longer sufficient. Additional security mechanisms are required to inspect email traffic, detect malicious content, and enforce security policies before messages reach end users.

A Mail Security Gateway provides an effective solution by acting as an intermediary between email clients and the internal mail server. Instead of allowing messages to be delivered directly, the gateway inspects every email using multiple security components before deciding whether the message should be accepted or rejected. This layered approach improves the overall security of the email infrastructure while maintaining normal communication for legitimate users.

1.2 Problem Statement

Traditional mail servers are primarily designed to send and receive email messages. Without additional security controls, they may allow spam, malicious attachments, or unauthorized SMTP activity to reach internal users.

Many organizations therefore deploy dedicated mail security solutions to inspect incoming email before delivery. However, commercial email security products can be expensive and may not be suitable for small organizations, educational institutions, or laboratory environments.

This project addresses that challenge by implementing a Mail Security Gateway using open-source technologies. The gateway is designed to inspect incoming email, detect malicious attachments, restrict unauthorized SMTP activity, and forward only legitimate messages to the Internal Mail Server.

1.3 Project Objectives

The primary objective of this internship project was to design, implement, and validate a secure Mail Security Gateway within a virtual laboratory environment.

The specific objectives were:

- Deploy an Internal Mail Server using Postfix.
 - Deploy a dedicated Mail Security Gateway.
 - Configure secure SMTP relay between the gateway and the Internal Mail Server.
 - Integrate Rspamd for spam analysis.
 - Integrate ClamAV for malware detection.
 - Configure UFW to strengthen host-level security.
 - Configure Fail2Ban to monitor SMTP-related activities.
 - Validate legitimate email delivery.
 - Validate malware detection using the EICAR antivirus test file.
 - Evaluate gateway behaviour during controlled SMTP abuse testing.
-

1.4 Scope of the Project

The project was implemented within a controlled virtual laboratory using Oracle VirtualBox. Multiple virtual machines were deployed to simulate a secure email environment without affecting production systems.

The implementation focused on the complete deployment and integration of the following components:

- Ubuntu Client
- Windows Server 2019 Client
- Mail Security Gateway
- Internal Mail Server

The project included the installation, configuration, testing, troubleshooting, and validation of each component. Validation was performed using controlled test scenarios that demonstrated normal email communication, malware detection, and SMTP connection management.

Although the implemented environment reflects common practices used in enterprise email security, the project was designed as an educational proof-of-concept rather than a production deployment.

1.5 Methodology

The implementation followed a structured and incremental methodology throughout the internship. Instead of configuring all services simultaneously, each component was deployed and verified individually before being integrated into the complete email security architecture.

The overall implementation process consisted of the following phases:

1. Planning the laboratory environment.
2. Deploying the Internal Mail Server.
3. Deploying the Mail Security Gateway.
4. Integrating spam filtering and malware scanning.
5. Applying security hardening measures.
6. Performing functional validation.
7. Troubleshooting identified issues.
8. Documenting the completed implementation.

This phased approach reduced configuration complexity and simplified troubleshooting during system integration.

1.6 Expected Outcomes

The expected outcome of this project was the successful implementation of a functional Mail Security Gateway capable of protecting an Internal Mail Server from common email-based threats.

The completed system was expected to:

- Successfully relay legitimate email.
- Inspect every incoming message before delivery.
- Detect malicious attachments using ClamAV.
- Apply spam analysis through Rspamd.
- Restrict unauthorized SMTP relay.
- Improve the overall security of the email infrastructure.

Successful validation of these functions would demonstrate that multiple open-source technologies can be integrated to provide layered email security within a laboratory environment.

1.7 Organization of the Report

This report is organized into fifteen chapters.

- **Chapter 1** introduces the project, objectives, methodology, and scope.
 - **Chapter 2** presents an overview of the internship organization and technical environment.
 - **Chapter 3** describes the Mail Security Gateway project and implementation requirements.
 - **Chapter 4** explains the overall system architecture and laboratory design.
 - **Chapters 5 to 7** document the deployment of the Internal Mail Server, Mail Security Gateway, and security hardening process.
 - **Chapters 8 to 10** present the validation of legitimate email delivery, malware detection, and SMTP abuse resistance.
 - **Chapter 11** discusses the troubleshooting process and problem resolution.
 - **Chapter 12** evaluates the completed implementation.
 - **Chapter 13** outlines the limitations of the project and possible future improvements.
 - **Chapter 14** provides the overall conclusion.
-

1.8 Chapter Summary

This chapter introduced the background and motivation for implementing a Mail Security Gateway using open-source technologies. It defined the project objectives, scope, methodology, and expected outcomes while providing an overview of the report structure. These topics establish the foundation for the implementation and validation activities described in the following chapters.

Chapter 2

Organization Overview

2.1 Introduction

This chapter provides a brief overview of the organization where the internship was completed. Understanding the organization's background and technical environment helps explain the purpose of the internship project and how it aligns with the organization's operational objectives.

The internship was carried out at **Plexus Cloud**, where the primary focus was to gain practical experience in Linux system administration, cybersecurity, and infrastructure security through hands-on implementation of real-world technical tasks.

2.2 Organization Profile

Plexus Cloud is a technology-focused organization that provides solutions related to cloud infrastructure, Linux server administration, cybersecurity, virtualization, and network services. The organization emphasizes practical implementation and encourages interns to work directly with industry-standard open-source technologies rather than relying solely on theoretical knowledge.

During the internship, interns were introduced to real administrative tasks involving Linux systems, network services, security monitoring, and infrastructure deployment. This practical approach enabled the development of technical skills through implementation, testing, troubleshooting, and documentation.

2.3 Areas of Technical Focus

The organization works with a variety of technologies that support secure and reliable IT infrastructure. During the internship, the following technical areas were particularly relevant:

- Linux Server Administration
- Network Services
- Virtualization
- Cybersecurity
- System Monitoring
- Infrastructure Deployment
- Open-Source Security Solutions

These areas provided the technical foundation required for completing the assigned internship project.

2.4 Internship Environment

The internship followed a project-based learning approach in which each intern was assigned practical implementation tasks. Instead of observing existing systems, interns were expected to build, configure, test, and validate their own environments.

For this project, a complete virtual laboratory was created using Oracle VirtualBox. Multiple virtual machines were deployed to simulate an enterprise-style email infrastructure. This environment allowed the implementation and validation of security controls without affecting production systems.

The laboratory environment consisted of:

Component	Purpose
Ubuntu Client	Email Sender

Windows Server 2019	Secondary Email Client
GatewayVM	Mail Security Gateway
ubuntu1	Internal Mail Server

This virtual infrastructure provided a safe environment for deployment, testing, and troubleshooting throughout the internship.

2.5 Internship Project

As part of the internship activities, I was assigned the task of implementing a **Mail Security Gateway** capable of protecting an internal mail server from malicious email traffic.

The project involved integrating multiple open-source technologies to inspect incoming email before delivery. Rather than deploying a standalone mail server, the objective was to create a layered email security solution capable of:

- Receiving SMTP traffic.
- Filtering spam.
- Scanning attachments for malware.
- Enforcing security policies.
- Relaying legitimate email to the protected mail server.

The implementation required configuring and integrating Postfix, Rspamd, ClamAV, UFW, and Fail2Ban within a controlled virtual environment.

2.6 Skills Developed During the Internship

Throughout the internship, practical experience was gained in several technical areas, including:

- Linux server administration.
- Mail server deployment.
- SMTP communication.
- Email security.
- Firewall configuration.
- Malware detection.
- Service integration.

- System troubleshooting.
- Log analysis.
- Technical documentation.

These activities helped bridge the gap between academic learning and practical system administration.

2.7 Chapter Summary

This chapter introduced the organization where the internship was completed and described the technical environment in which the project was developed. It also outlined the internship objectives, the laboratory infrastructure, and the practical skills acquired throughout the implementation process. This organizational context provides the foundation for understanding the design and development of the Mail Security Gateway presented in the following chapters.

Chapter 3

Project Overview

3.1 Introduction

This chapter provides an overview of the internship project assigned during the training period. It explains the purpose of the project, the security challenges addressed, the implementation objectives, and the technologies selected to achieve the desired outcome.

The project focused on designing, implementing, and validating a secure Mail Security Gateway capable of inspecting incoming email traffic before forwarding legitimate messages to an Internal Mail Server. The implementation was carried out entirely within a virtual laboratory using open-source technologies.

3.2 Project Description

Email is one of the most frequently targeted services in modern network environments. Attackers commonly use email to distribute malware, deliver spam, perform phishing attacks, or exploit improperly configured mail servers.

To reduce these risks, organizations often deploy a Mail Security Gateway between external clients and the internal mail server. Rather than allowing messages to be delivered directly, the gateway acts as a security checkpoint where every email is inspected before reaching the recipient.

The internship project required the implementation of such a gateway using widely adopted open-source technologies. The completed system was expected to inspect email traffic, detect malicious attachments, enforce security policies, and relay only legitimate messages to the Internal Mail Server.

3.3 Project Objectives

The project was designed to achieve the following objectives:

Primary Objective

To implement a secure Mail Security Gateway capable of protecting an Internal Mail Server through layered email inspection and security enforcement.

Specific Objectives

- Deploy an Internal Mail Server.
- Deploy a dedicated Mail Security Gateway.
- Configure secure SMTP communication between systems.
- Integrate spam filtering.
- Integrate antivirus scanning.
- Apply host-level security hardening.
- Validate normal email communication.
- Validate malware detection.
- Evaluate gateway behaviour during abnormal SMTP activity.

These objectives guided the implementation throughout the internship.

3.4 Project Scope

The implementation was limited to a controlled laboratory environment and focused on practical deployment rather than production operation.

The project included:

- Virtual machine deployment.
- Network configuration.
- Mail server installation.
- Gateway configuration.
- Spam filtering.
- Malware scanning.
- Firewall configuration.
- SMTP security.
- Functional testing.
- Validation.
- Troubleshooting.
- Technical documentation.

The project did not include:

- Public Internet deployment.
- Enterprise high-availability configuration.
- Cloud deployment.
- Performance benchmarking under production workloads.

Limiting the scope allowed the project to focus on the successful integration and validation of the core security components.

3.5 Development Environment

A virtual laboratory was created using Oracle VirtualBox to simulate a secure enterprise email environment.

The laboratory consisted of four virtual machines.

Virtual Machine	Role
Ubuntu Client	Email Sender

Windows Server 2019	Secondary Email Client
GatewayVM	Mail Security Gateway
ubuntu1	Internal Mail Server

The systems communicated through an isolated Host-Only network, allowing secure testing without affecting external systems.

Figure 3.1 Virtual Laboratory Environment



3.6 Technologies Used

The implementation relied entirely on open-source software.

Technology	Purpose
Ubuntu Server 22.04 LTS	Operating System
Oracle VirtualBox	Virtualization Platform
Postfix	Mail Transfer Agent (SMTP)

Dovecot	Mail Access Service
Rspamd	Spam Filtering Engine
ClamAV	Antivirus Engine
UFW	Host Firewall
Fail2Ban	Intrusion Prevention
Mail Utility / Mutt	Email Testing
PowerShell	Windows Email Testing

Each component played a specific role within the overall email security architecture.

3.7 Expected System Architecture

The expected architecture consisted of a layered mail flow where every incoming email passed through the Mail Security Gateway before reaching the Internal Mail Server.

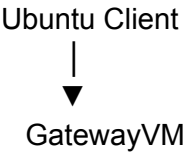
The Mail Security Gateway was responsible for:

- Receiving SMTP traffic.
- Inspecting email headers and content.
- Scanning attachments for malware.
- Applying security policies.
- Forwarding legitimate messages.
- Rejecting malicious email.

This design ensured that the Internal Mail Server was protected from common email-based threats.

Figure 3.2 Expected Mail Flow

Insert Diagram



(Postfix + Rspamd + ClamAV)



Accept / Reject



ubuntu1

(Internal Mail Server)

3.8 Project Deliverables

By the end of the internship, the completed implementation was expected to demonstrate:

- Successful deployment of the Internal Mail Server.
- Functional Mail Security Gateway.
- Secure SMTP relay.
- Spam filtering.
- Malware detection.
- Basic host-level security hardening.
- Validation of legitimate email delivery.
- Validation of malware detection using the EICAR test file.
- Validation of controlled SMTP abuse testing.
- Comprehensive technical documentation.

These deliverables served as measurable indicators of the project's successful completion.

3.9 Chapter Summary

This chapter introduced the internship project, its objectives, scope, development environment, and expected deliverables. It also described the technologies selected for the implementation and presented the planned system architecture that guided the development of the Mail Security Gateway. The following chapter explains the overall system design and network architecture in greater detail before the implementation process begins.

Chapter 4

System Design

4.1 Introduction

Before implementing the Mail Security Gateway, a suitable system architecture was designed to ensure secure communication between client systems and the Internal Mail Server. The architecture was developed to provide multiple layers of protection while maintaining normal email delivery for legitimate users.

This chapter presents the overall system design, including the virtual laboratory environment, network topology, component interactions, and email processing workflow. Understanding the system design provides the foundation for the implementation described in the following chapters.

4.2 System Architecture

The implemented solution follows a layered architecture where all incoming email traffic passes through a dedicated Mail Security Gateway before reaching the Internal Mail Server.

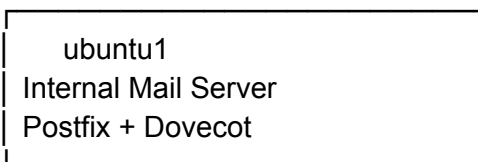
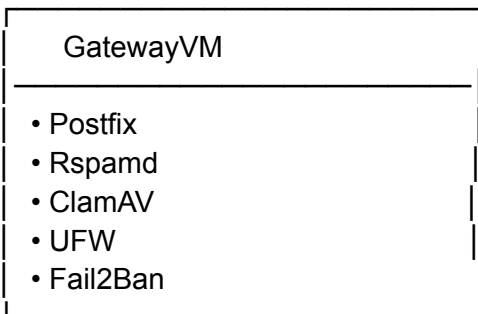
Instead of allowing client systems to communicate directly with the mail server, every email is first inspected by the gateway. The gateway performs spam analysis, malware scanning, and security policy enforcement before deciding whether the message should be forwarded or rejected.

This approach reduces the exposure of the Internal Mail Server to malicious email traffic and improves the overall security of the email infrastructure.

Figure 4.1 Overall System Architecture

Ubuntu Client

Windows Server 2019



4.3 Virtual Laboratory Environment

The project was implemented in a virtual laboratory using Oracle VirtualBox. Virtualization made it possible to deploy multiple servers and client systems on a single physical computer while maintaining network isolation.

Using virtual machines also simplified testing, troubleshooting, and system recovery throughout the implementation.

The laboratory consisted of four virtual machines, each assigned a specific responsibility within the email infrastructure.

Virtual Machine	Operating System	Purpose
Ubuntu Client	Ubuntu Desktop	Sends email
Windows Server 2019	Windows Server	Secondary email client
GatewayVM	Ubuntu Server	Mail Security Gateway
ubuntu1	Ubuntu Server	Internal Mail Server

Figure 4.2 Virtual Laboratory



4.4 Network Topology

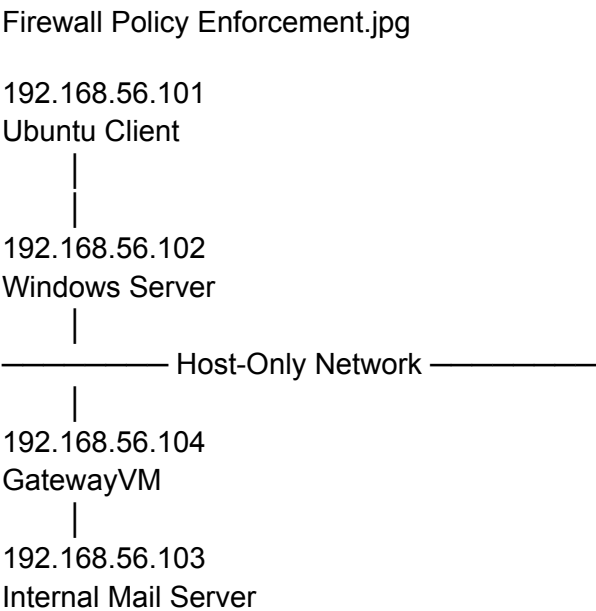
All virtual machines communicated through an isolated **Host-Only Network**, ensuring that testing remained within the laboratory environment.

Each virtual machine was assigned a static IP address to simplify SMTP communication and system administration.

System	Operating System	Purpose
Ubuntu Client	192.168.56.101	Sends email
Windows Server 2019	192.168.56.102	Secondary email client
GatewayVM	192.168.56.104	Mail Security Gateway
ubuntu1	192.168.56.103	Internal Mail Server

The isolated network allowed realistic testing without exposing the systems to external traffic.

Figure 4.3 Network Topology



4.5 Email Processing Workflow

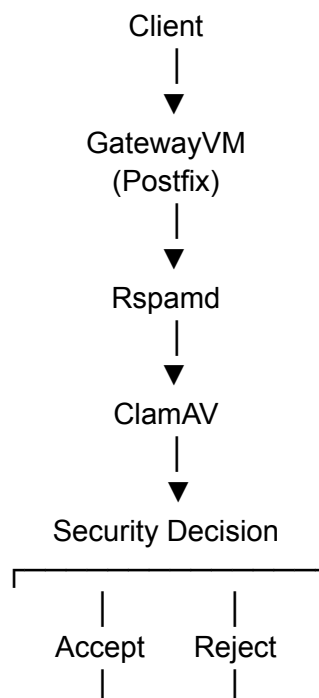
When an email is sent by a client, it does not immediately reach the Internal Mail Server.

Instead, the message passes through several security stages within the Mail Security Gateway. The processing workflow is as follows:

1. Client sends an email.
2. GatewayVM receives the SMTP connection.
3. Postfix accepts the message.
4. Rspamd performs spam analysis.
5. ClamAV scans attachments for malware.
6. Security policies are evaluated.
7. Legitimate messages are forwarded.
8. Malicious messages are rejected.

This layered workflow ensures that only safe email reaches the recipient.

Figure 4.4 Email Processing Workflow



4.6 Security Components

Several open-source technologies were integrated to provide layered protection.

Component	Function
Postfix	SMTP relay and mail transfer
Rspamd	Spam analysis and filtering
ClamAV	Malware detection
UFW	Firewall protection
Fail2Ban	Detection of suspicious SMTP activity
Dovecot	Mail access service

Each component performs a dedicated function while working together to protect the Internal Mail Server.

4.7 Design Considerations

Several factors influenced the system design:

- Separation of security functions from the Internal Mail Server.
- Layered inspection of incoming email.
- Use of open-source technologies.
- Isolated laboratory environment for safe testing.
- Modular architecture allowing independent troubleshooting.

These considerations improved both the security and maintainability of the implementation.

4.8 Advantages of the Proposed Architecture

The selected architecture provides several practical benefits:

- Centralized inspection of all incoming email.
- Reduced exposure of the Internal Mail Server.
- Multiple security layers for spam and malware detection.
- Easier monitoring and troubleshooting.
- Flexible expansion for future enhancements.

Although implemented within a laboratory environment, the architecture demonstrates the principles commonly used in secure email infrastructures.

4.9 Chapter Summary

This chapter presented the overall design of the Mail Security Gateway and described the virtual laboratory, network topology, system architecture, and email processing workflow. The layered design ensures that every incoming email is inspected before delivery to the Internal Mail Server, providing a strong foundation for the implementation discussed in the following chapters.

Chapter 5

Internal Mail Server Deployment

5.1 Introduction

The first stage of the implementation was the deployment of the Internal Mail Server. This server acts as the final destination for all legitimate email messages after they have been inspected by the Mail Security Gateway.

Rather than exposing the mail server directly to client systems, it was placed behind the gateway to provide an additional layer of security. This design ensures that every incoming email is examined before it reaches the recipient's mailbox.

This chapter describes the installation, configuration, and validation of the Internal Mail Server.

5.2 Preparing the Server Environment

A dedicated Ubuntu Server virtual machine named **ubuntu1** was prepared to function as the Internal Mail Server.

Before installing the mail services, the operating system was updated to ensure that all packages were current.

The following tasks were performed:

- Updated the package repository.
- Upgraded installed packages.
- Verified network connectivity.
- Confirmed the assigned static IP address.
- Verified hostname configuration.

Preparing the operating system before installing mail services reduced compatibility issues during deployment.

Figure 5.1 Ubuntu Server Initial Configuration

```
ubuntums@ubuntu1:~$ grep -E "(myhostname|mydomain|mydestination|inet_
interfaces|home_mailbox)" /etc/postfix/main.cf
myhostname = ubuntu1
mydestination = $myhostname, mail.lab, ubuntu1, localhost.localdomain,
localhost
inet_interfaces = all
ubuntums@ubuntu1:~$ hostnamectl
Static hostname: ubuntu1
Icon name: computer-vm
Chassis: vm
Machine ID: a5e65294095e4f2d9c2fe6a18b179c26
Boot ID: f74d64a920b4467fb394e95f8ea0ea17
Virtualization: oracle
Operating System: Ubuntu 22.04.5 LTS
Kernel: Linux 5.15.0-181-generic
Architecture: x86-64
Hardware Vendor: innotek GmbH
Hardware Model: VirtualBox
ubuntums@ubuntu1:~$ hostname -I
10.0.2.15 192.168.56.103 fd17:625c:f037:2:a00:27ff:fe9f:f847
ubuntums@ubuntu1:~$
```

5.3 Installing Postfix

Postfix was selected as the Mail Transfer Agent (MTA) because it is stable, secure, lightweight, and widely used in Linux-based email systems.

During installation, Postfix was configured to operate as an **Internet Site**, allowing the server to receive and process email for the local domain.

After installation, the service was verified to ensure that it started successfully.

The successful installation of Postfix established the SMTP functionality required for the Internal Mail Server.

Figure 5.2 Postfix Installation

```
ubuntums@ubuntu1:~$ systemctl status postfix
● postfix.service - Postfix Mail Transport Agent
   Loaded: loaded (/lib/systemd/system/postfix.service; >
   Active: active (exited) since Tue 2026-06-02 09:25:34>
     Docs: man:postfix(1)
    Main PID: 3979 (code=exited, status=0/SUCCESS)
      CPU: 1ms
lines 1-6/6 (END)
```

5.4 Configuring Postfix

After installation, the Postfix configuration file (`main.cf`) was modified to support the laboratory environment.

The configuration included:

- Mail hostname.
- Local domain.
- Trusted network configuration.
- Mail destination settings.
- SMTP listening interface.

These settings enabled the server to receive email from the Mail Security Gateway while preventing unnecessary exposure to external systems.

After completing the configuration, the Postfix service was restarted to apply the changes.

Figure 5.3 Postfix Configuration

```
ubuntums@ubuntu1:~$ grep -E "^(myhostname|mydomain|mydestination|inet_
interfaces|home_mailbox)" /etc/postfix/main.cf
myhostname = ubuntu1
mydestination = $myhostname, mail.lab, ubuntu1, localhost.localdomain,
localhost
inet_interfaces = all
ubuntums@ubuntu1:~$
```

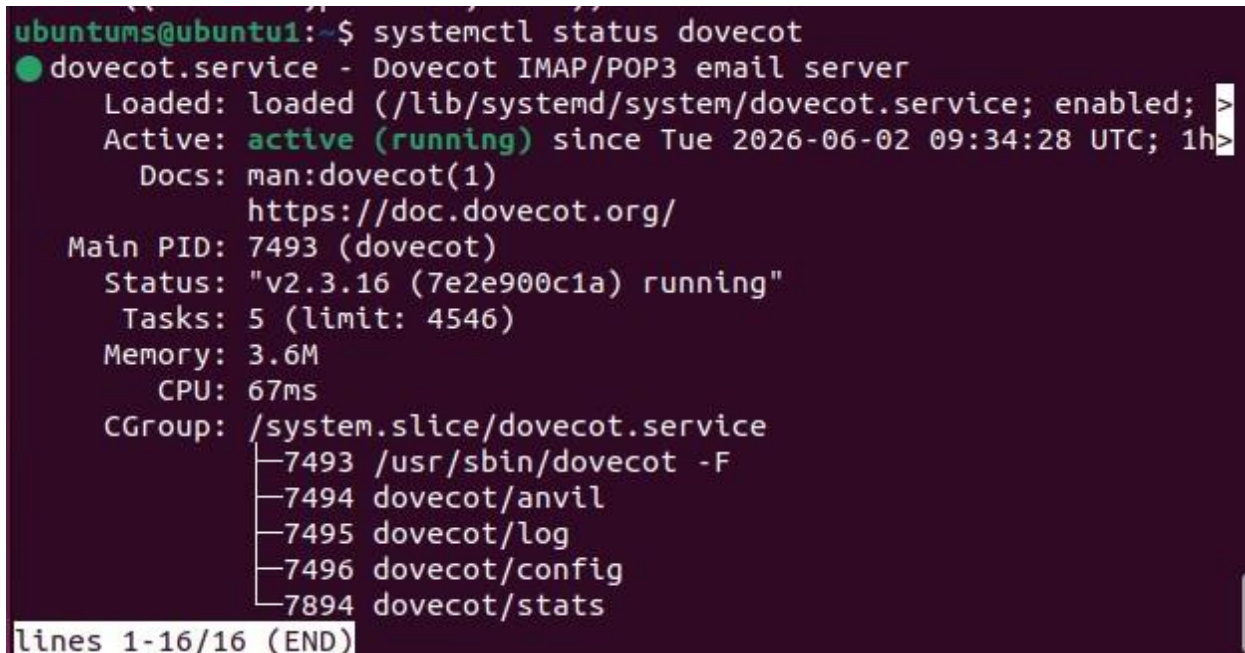
5.5 Installing Dovecot

To allow users to access their mailboxes, Dovecot was installed as the Mail Delivery Agent (MDA) and IMAP/POP3 server.

Dovecot provides secure mailbox access and works alongside Postfix to complete the email delivery process.

Once installed, the service was started and verified to ensure proper operation.

Figure 5.4 Dovecot Installation



```
ubuntums@ubuntu1:~$ systemctl status dovecot
● dovecot.service - Dovecot IMAP/POP3 email server
   Loaded: loaded (/lib/systemd/system/dovecot.service; enabled; >
   Active: active (running) since Tue 2026-06-02 09:34:28 UTC; 1h>
     Docs: man:dovecot(1)
           https://doc.dovecot.org/
   Main PID: 7493 (dovecot)
    Status: "v2.3.16 (7e2e900c1a) running"
     Tasks: 5 (limit: 4546)
    Memory: 3.6M
       CPU: 67ms
    CGroup: /system.slice/dovecot.service
            └─7493 /usr/sbin/dovecot -F
              └─7494 dovecot/anvil
                └─7495 dovecot/log
                  └─7496 dovecot/config
                    └─7894 dovecot/stats
lines 1-16/16 (END)
```

5.6 Creating the Mail User

A dedicated mailbox user was created for validation purposes.

The user account allowed the successful delivery and verification of legitimate email messages during later testing phases.

Creating a dedicated mailbox simplified validation by providing a consistent destination for all email sent through the Mail Security Gateway.

Figure 5.5 Mail User Creation

```
ubuntums@ubuntu1:~$ id mailuser
uid=1001(mailuser) gid=1001(mailuser) groups=1001(mailuser)
ubuntums@ubuntu1:~$
```

5.7 Service Verification

After completing the installation and configuration, all mail services were verified.

The following services were checked:

Service	Status
Postfix	Running
Dovecot	Running

In addition to service verification, basic network connectivity tests confirmed that the Internal Mail Server was reachable from the Mail Security Gateway.

These checks ensured that the server was ready for integration with the remaining components.

Figure 5.6 Service Verification

```
ir@GatewayVM:~$ sudo systemctl status postfix
[sudo] password for ir:
● postfix.service - Postfix Mail Transport Agent
   Loaded: loaded (/lib/systemd/system/postfix.service; enabled; ve>
   Active: active (exited) since Fri 2026-06-26 18:00:39 UTC; 2h 44>
   Docs: man:postfix(1)
   Process: 1700 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
   Process: 2847 ExecReload=/bin/true (code=exited, status=0/SUCCESS)
   Main PID: 1700 (code=exited, status=0/SUCCESS)
   CPU: 1ms

Jun 26 18:00:39 GatewayVM systemd[1]: Starting Postfix Mail Transport>
Jun 26 18:00:39 GatewayVM systemd[1]: Finished Postfix Mail Transport>
Jun 26 18:02:38 GatewayVM systemd[1]: Reloading Postfix Mail Transport>
Jun 26 18:02:38 GatewayVM systemd[1]: Reloaded Postfix Mail Transport>
ir@GatewayVM:~$ sudo systemctl status clamav-daemon
● clamav-daemon.service - Clam AntiVirus userspace daemon
   Loaded: loaded (/lib/systemd/system/clamav-daemon.service; enabl>
   Drop-In: /etc/systemd/system/clamav-daemon.service.d
           └─extend.conf
   Active: active (running) since Fri 2026-06-26 18:00:15 UTC; 2h 4>
   TriggeredBy: ● clamav-daemon.socket
   Docs: man:clamd(8)
          man:clamd.conf(5)
          https://docs.clamav.net/
   Process: 790 ExecStartPre=/bin/mkdir -p /run/clamav (code=exited,>
   Process: 812 ExecStartPre=/bin/chown clamav /run/clamav (code=exi>
   Main PID: 818 (clamd)
   Tasks: 2 (limit: 3415)
   Memory: 1.0G
   CPU: 11.620s
   CGroup: /system.slice/clamav-daemon.service
           └─818 /usr/sbin/clamd --foreground=true

Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> PDF>
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> SWF>
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> HTM>
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> XML>
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> HWP>
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> One>
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> Sel>
ir@GatewayVM:~$ sudo systemctl status rspamd
● rspamd.service - rapid spam filtering system
   Loaded: loaded (/lib/systemd/system/rspamd.service; enabled; ven>
   Active: active (running) since Fri 2026-06-26 18:00:26 UTC; 2h 4>
   Docs: https://rspamd.com/doc/
   Main PID: 1048 (rspamd)
```

5.8 Internal Mail Server Role

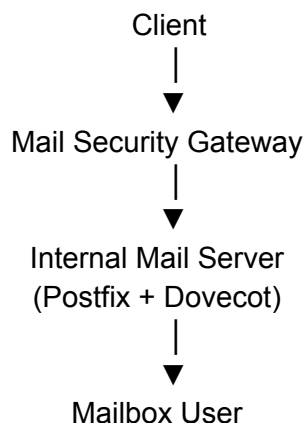
Within the final architecture, the Internal Mail Server performs only one responsibility: storing and delivering legitimate email to local users.

Unlike the Mail Security Gateway, the Internal Mail Server does not perform spam filtering, malware detection, or security policy enforcement.

These responsibilities are intentionally separated to improve security, simplify administration, and reduce the exposure of the mail server to malicious traffic.

This separation of responsibilities forms the foundation of the layered architecture introduced in Chapter 4.

Figure 5.7 Internal Mail Server in the Architecture



5.9 Chapter Summary

This chapter described the deployment of the Internal Mail Server using Ubuntu Server, Postfix, and Dovecot. The server was prepared, configured, and verified to provide a secure destination for legitimate email messages. With the Internal Mail Server successfully deployed, the next

stage of the implementation focuses on configuring the Mail Security Gateway, which is responsible for inspecting and filtering email before delivery.

Chapter 6

Mail Security Gateway Deployment

6.1 Introduction

After successfully deploying the Internal Mail Server, the next stage of the project was the implementation of the Mail Security Gateway. The gateway acts as the first point of contact for all incoming email and serves as the primary security layer within the email infrastructure.

Instead of allowing client systems to communicate directly with the Internal Mail Server, all email traffic is routed through the gateway. The gateway is responsible for receiving SMTP connections, inspecting messages, enforcing security policies, and forwarding only legitimate email to the Internal Mail Server.

This chapter describes the deployment and configuration of the Mail Security Gateway.

6.2 Preparing GatewayVM

A dedicated Ubuntu Server virtual machine named **GatewayVM** was prepared to function as the Mail Security Gateway.

Before installing the required security services, the operating system was updated and basic network connectivity was verified.

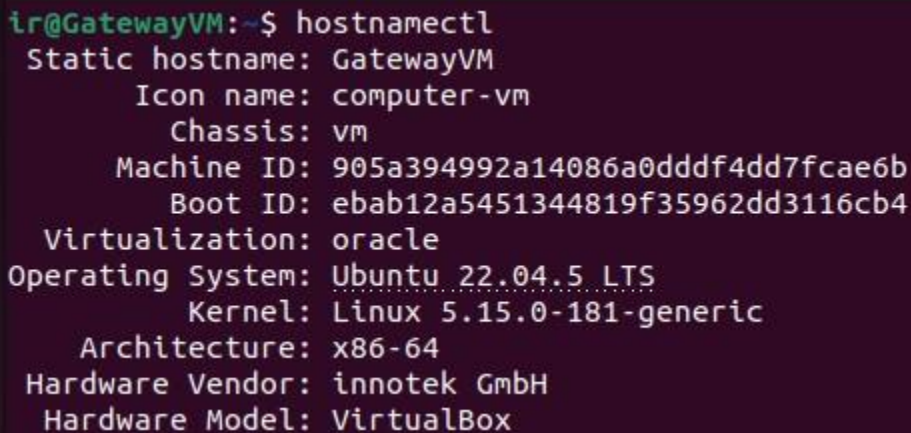
The preparation process included:

- Updating the operating system.
- Installing required software packages.

- Configuring the hostname.
- Verifying the static IP address.
- Confirming communication with the Internal Mail Server.

Completing these initial tasks ensured that the server was ready for the installation of the gateway services.

Figure 6.1 GatewayVM Initial Configuration



```
lr@GatewayVM:~$ hostnamectl
Static hostname: GatewayVM
    Icon name: computer-vm
    Chassis: vm
    Machine ID: 905a394992a14086a0dddf4dd7fcae6b
    Boot ID: ebab12a5451344819f35962dd3116cb4
    Virtualization: oracle
Operating System: Ubuntu 22.04.5 LTS
    Kernel: Linux 5.15.0-181-generic
    Architecture: x86-64
Hardware Vendor: innotek GmbH
Hardware Model: VirtualBox
```

6.3 Installing Postfix on the Gateway

Postfix was installed on GatewayVM to receive SMTP connections from client systems and relay legitimate email to the Internal Mail Server.

Unlike the Internal Mail Server, the gateway was configured to act as an SMTP relay rather than a mailbox server.

Its primary responsibilities include:

- Receiving email.
- Processing SMTP requests.
- Relaying approved messages.
- Rejecting malicious traffic.

After installation, the Postfix service was verified to ensure successful operation.

Figure 6.2 Gateway Postfix Installation

```
ubuntu@ubuntu1:~$ systemctl status postfix
● postfix.service - Postfix Mail Transport Agent
   Loaded: loaded (/lib/systemd/system/postfix.service; >
   Active: active (exited) since Tue 2026-06-02 09:25:34>
     Docs: man:postfix(1)
    Main PID: 3979 (code=exited, status=0/SUCCESS)
      CPU: 1ms
lines 1-6/6 (END)
```

6.4 Configuring SMTP Relay

The Postfix configuration on GatewayVM was modified to relay approved email to the Internal Mail Server.

The configuration included:

- Defining the relay destination.
- Configuring trusted networks.
- Setting the hostname.
- Configuring SMTP interfaces.
- Restricting unauthorized relay.

Proper relay configuration ensured that legitimate email could reach the Internal Mail Server while preventing unauthorized SMTP usage.

Figure 6.3 SMTP Relay Configuration

```
ir@GatewayVM:~$ grep -E "(myhostname|relayhost|mydestination|relay_do
mains)" /etc/postfix/main.cf
myhostname = GatewayVM
mydestination = $myhostname, gateway.lab, GatewayVM, localhost.localdo
main, localhost
relayhost = [192.168.56.103]:25
ir@GatewayVM:~$
```

6.5 Testing Mail Flow

After configuring the relay service, communication between GatewayVM and the Internal Mail Server was verified.

Test emails were generated from the client system and forwarded through the gateway.

Successful delivery confirmed that:

- SMTP communication was functioning.
- Relay configuration was correct.
- GatewayVM successfully forwarded messages.

This validation established the foundation required before introducing spam filtering and malware scanning.

Figure 6.4 Successful Mail Relay



```
Jun 26 19:32:39 gateway postfix/smtp[9393]: 9A3A642440: to=<mailuser@u  
buntu1>, relay=192.168.56.103[192.168.56.103]:25, delay=1136, delays=1  
135/0.01/0.13/0.02, dsn=2.0.0, status=sent (250 2.0.0 Ok: queued as 3B  
1941606ED)
```

6.6 Installing Rspamd

Rspamd was installed to provide spam filtering and email analysis capabilities.

Rspamd evaluates incoming messages using multiple criteria, including:

- Message headers.
- Sender reputation.
- Content analysis.
- Email structure.
- Authentication information.

Based on these evaluations, Rspamd assigns a score that helps determine whether a message should be accepted or rejected.

Integrating Rspamd significantly improved the gateway's ability to identify suspicious email.

Figure 6.5 Rspamd Installation

```
ir@GatewayVM:~$ sudo systemctl status rspamd
● rspamd.service - rapid spam filtering system
   Loaded: loaded (/lib/systemd/system/rspamd.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2026-06-26 18:00:26 UTC; 2h 2min ago
     Docs: https://rspamd.com/doc/
    Main PID: 1048 (rspamd)
      Tasks: 5 (limit: 3415)
    Memory: 120.9M
       CPU: 2.499s
    CGroup: /system.slice/rspamd.service
            └─1048 "rspamd: main process; 2.0 msg/sec, 0.0 msg/sec"
              └─1731 "rspamd: rspamd_proxy process (localhost:11332)"
                └─1732 "rspamd: controller process (localhost:11334)"
                  └─1733 "rspamd: normal process (localhost:11333)"
                    └─1734 "rspamd: hs_helper process"

Jun 26 18:00:26 GatewayVM systemd[1]: Started rapid spam filtering system.
lines 1-16/16 (END)
```

6.7 Integrating ClamAV

To detect malicious attachments, ClamAV was integrated with Rspamd.

Whenever an email contains an attachment, ClamAV scans the file using its antivirus signatures. If malware is detected, Rspamd receives the scan result and instructs Postfix to reject the email before it reaches the Internal Mail Server.

This integration provides an additional security layer capable of preventing malware delivery through email.

Figure 6.6 ClamAV Integration


```

lr@GatewayVM:~$ sudo systemctl status clamav-daemon
● clamav-daemon.service - Clam AntiVirus userspace daemon
   Loaded: loaded (/lib/systemd/system/clamav-daemon.service; enabled; vendor preset: enabled)
   Drop-In: /etc/systemd/system/clamav-daemon.service.d
            └─extend.conf
   Active: active (running) since Fri 2026-06-26 18:00:15 UTC; 2h 14min ago
   TriggeredBy: ● clamav-daemon.socket
     Docs: man:clamd(8)
            man:clamd.conf(5)
            https://docs.clamav.net/
   Process: 790 ExecStartPre=/bin/mkdir -p /run/clamav (code=exited, status=0/SUCCESS)
   Process: 812 ExecStartPre=/bin/chown clamav /run/clamav (code=exited, status=0/SUCCESS)
  Main PID: 818 (clamd)
    Tasks: 2 (limit: 3415)
   Memory: 1.0G
      CPU: 11.620s
   CGroup: /system.slice/clamav-daemon.service
            └─818 /usr/sbin/clamd --foreground=true

Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> OLE>
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> PDF>
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> SWF>
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> HTM>
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> XML>
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> HWP>
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> One>
lines 1-25...skipping...
● clamav-daemon.service - Clam AntiVirus userspace daemon
   Loaded: loaded (/lib/systemd/system/clamav-daemon.service; enabled; vendor preset: enabled)
   Drop-In: /etc/systemd/system/clamav-daemon.service.d
            └─extend.conf
   Active: active (running) since Fri 2026-06-26 18:00:15 UTC; 2h 14min ago
   TriggeredBy: ● clamav-daemon.socket
     Docs: man:clamd(8)
            man:clamd.conf(5)
            https://docs.clamav.net/
   Process: 790 ExecStartPre=/bin/mkdir -p /run/clamav (code=exited, status=0/SUCCESS)
   Process: 812 ExecStartPre=/bin/chown clamav /run/clamav (code=exited, status=0/SUCCESS)
  Main PID: 818 (clamd)
    Tasks: 2 (limit: 3415)
   Memory: 1.0G
      CPU: 11.620s
   CGroup: /system.slice/clamav-daemon.service
            └─818 /usr/sbin/clamd --foreground=true

```

6.8 Gateway Processing Workflow

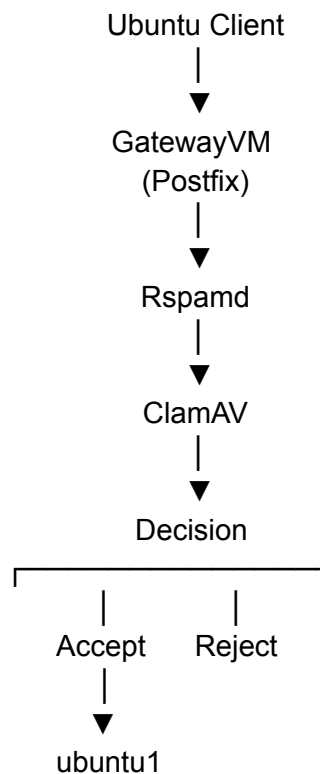
After all services were integrated, the Mail Security Gateway processed incoming email according to the following sequence:

1. Client establishes an SMTP connection.
2. GatewayVM receives the message.
3. Postfix accepts the SMTP transaction.
4. Rspamd analyzes the email.
5. ClamAV scans all attachments.

6. Security policies are evaluated.
7. Legitimate email is forwarded.
8. Malicious email is rejected.

This workflow ensures that all incoming messages undergo inspection before delivery.

Figure 6.7 Gateway Processing Workflow



6.9 Gateway Verification

Following the installation and integration of all components, GatewayVM was tested to ensure that every service operated correctly.

The following components were verified:

Component	Status
-----------	--------

Postfix	Running
Rspamd	Running
ClamAV	Running

Mail flow testing confirmed that GatewayVM successfully received SMTP traffic, inspected messages, and forwarded approved email to the Internal Mail Server.

Figure 6.8 Gateway Service Verification

```
ir@GatewayVM:~$ sudo systemctl status postfix
● postfix.service - Postfix Mail Transport Agent
   Loaded: loaded (/lib/systemd/system/postfix.service; enabled; ve>
   Active: active (exited) since Fri 2026-06-26 18:00:39 UTC; 2h 14>
   Docs: man:postfix(1)
   Process: 1700 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
   Process: 2847 ExecReload=/bin/true (code=exited, status=0/SUCCESS)
  Main PID: 1700 (code=exited, status=0/SUCCESS)
     CPU: 1ms

Jun 26 18:00:39 GatewayVM systemd[1]: Starting Postfix Mail Transport>
Jun 26 18:00:39 GatewayVM systemd[1]: Finished Postfix Mail Transport>
Jun 26 18:02:38 GatewayVM systemd[1]: Reloading Postfix Mail Transpor>
Jun 26 18:02:38 GatewayVM systemd[1]: Reloaded Postfix Mail Transport>
ir@GatewayVM:~$ sudo systemctl status rspamd
● rspamd.service - rapid spam filtering system
   Loaded: loaded (/lib/systemd/system/rspamd.service; enabled; ven>
   Active: active (running) since Fri 2026-06-26 18:00:26 UTC; 2h 1>
   Docs: https://rspamd.com/doc/
  Main PID: 1048 (rspamd)
    Tasks: 5 (limit: 3415)
   Memory: 121.0M
      CPU: 2.411s
   CGroup: /system.slice/rspamd.service
           └─1048 "rspamd: main process; 2.0 msg/sec, 0.0 msg/sec s>
             └─1731 "rspamd: rspamd_proxy process (localhost:11332)"
               └─1732 "rspamd: controller process (localhost:11334)" ""
                 └─1733 "rspamd: normal process (localhost:11333)" "" "" >
                   └─1734 "rspamd: hs_helper process" "" "" "" "" "" "" "" >

Jun 26 18:00:26 GatewayVM systemd[1]: Started rapid spam filtering sy>
ir@GatewayVM:~$ sudo systemctl status clamav-daemon
● clamav-daemon.service - Clam AntiVirus userspace daemon
   Loaded: loaded (/lib/systemd/system/clamav-daemon.service; enabl>
   Drop-In: /etc/systemd/system/clamav-daemon.service.d
            └─extend.conf
   Active: active (running) since Fri 2026-06-26 18:00:15 UTC; 2h 1>
  TriggeredBy: ● clamav-daemon.socket
     Docs: man:clamd(8)
           man:clamd.conf(5)
           https://docs.clamav.net/
   Process: 790 ExecStartPre=/bin/mkdir -p /run/clamav (code=exited,>
   Process: 812 ExecStartPre=/bin/chown clamav /run/clamav (code=exi>
  Main PID: 818 (clamd)
    Tasks: 2 (limit: 3415)
   Memory: 1.0G
      CPU: 11.620s
```

6.10 Chapter Summary

This chapter described the deployment of the Mail Security Gateway using Ubuntu Server and Postfix. The gateway was configured to relay email to the Internal Mail Server while integrating Rspamd for spam analysis and ClamAV for malware detection. Successful verification confirmed that GatewayVM was capable of inspecting incoming email and enforcing security decisions before forwarding legitimate messages. With the gateway fully operational, the next chapter focuses on strengthening the overall security of the environment through firewall configuration, intrusion prevention, and additional hardening measures.

Chapter 7

Security Hardening of the Mail Security Gateway

7.1 Introduction

Deploying a Mail Security Gateway alone does not provide sufficient protection against modern email-based threats. After the gateway services were successfully deployed, additional security measures were implemented to strengthen the overall infrastructure and improve its resistance to unauthorized access and malicious activity.

The security hardening process focused on protecting the gateway itself rather than the email messages alone. Multiple security mechanisms were integrated to control network access, detect malware, monitor suspicious SMTP activity, and improve the overall security posture of the system.

This chapter describes the implementation of these security measures and explains their role within the Mail Security Gateway.

7.2 Security Hardening Objectives

The primary objectives of the security hardening phase were:

- Restrict unnecessary network access.
- Protect the gateway from unauthorized connections.
- Detect malicious email attachments.
- Monitor suspicious SMTP activities.
- Improve the overall security of the Mail Security Gateway.

Implementing these measures created multiple defensive layers around the gateway before beginning the final validation process.

7.3 Firewall Configuration Using UFW

The **Uncomplicated Firewall (UFW)** was configured to control network access to the Mail Security Gateway.

Rather than allowing unrestricted communication, only the services required for the operation of the mail infrastructure were permitted.

The firewall configuration ensured that unnecessary network ports remained closed while allowing legitimate SMTP communication between the client systems, GatewayVM, and the Internal Mail Server.

Restricting network access reduces the attack surface and prevents unauthorized connections to the gateway.

Figure 7.1 UFW Firewall Configuration


```

Rules updated
Rules updated (v6)
ubuntums@ubuntu1:~$ sudo ufw enable
Firewall is active and enabled on system startup
ubuntums@ubuntu1:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
25/tcp ALLOW IN Anywhere
143/tcp ALLOW IN Anywhere
993/tcp ALLOW IN Anywhere
995/tcp ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)
25/tcp (v6) ALLOW IN Anywhere (v6)
143/tcp (v6) ALLOW IN Anywhere (v6)
993/tcp (v6) ALLOW IN Anywhere (v6)
995/tcp (v6) ALLOW IN Anywhere (v6)
ubuntums@ubuntu1:~$

```

7.4 Malware Protection Using ClamAV

Although ClamAV was integrated with Rspamd during the gateway deployment, its primary role as an antivirus engine forms an important part of the security hardening process.

Whenever an email containing an attachment is received, ClamAV automatically scans the attachment using its virus signature database.

If a malicious file is detected, ClamAV reports the result to Rspamd, which instructs Postfix to reject the email before it reaches the Internal Mail Server.

This layered approach prevents malicious attachments from being delivered to end users.

Figure 7.2 ClamAV Service Verification

```
lr@GatewayVM:~$ sudo systemctl status clamav-daemon
● clamav-daemon.service - Clam AntiVirus userspace daemon
   Loaded: loaded (/lib/systemd/system/clamav-daemon.service; enabled; vendor preset: enabled)
   Drop-In: /etc/systemd/system/clamav-daemon.service.d
            └─extend.conf
   Active: active (running) since Fri 2026-06-26 18:00:15 UTC; 2h 1min ago
   TriggeredBy: ● clamav-daemon.socket
     Docs: man:clamd(8)
           man:clamd.conf(5)
           https://docs.clamav.net/
   Process: 790 ExecStartPre=/bin/mkdir -p /run/clamav (code=exited, status=0/SUCCESS)
   Process: 812 ExecStartPre=/bin/chown clamav /run/clamav (code=exited, status=0/SUCCESS)
  Main PID: 818 (clamd)
    Tasks: 2 (limit: 3415)
   Memory: 1.0G
      CPU: 11.620s
   CGroup: /system.slice/clamav-daemon.service
           └─818 /usr/sbin/clamd --foreground=true

Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> OLE
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> PDF
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> SWF
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> HTM
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> XML
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> HWP
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> One
```

7.5 Spam Filtering Using Rspamd

Rspamd was configured to analyze every incoming email before delivery.

Instead of relying on a single rule, Rspamd evaluates multiple characteristics of each message, including:

- Email headers.
- Message content.
- Sender information.
- Attachment properties.
- Authentication results.

Based on the analysis, Rspamd calculates a score and determines whether the message should be accepted or rejected.

This additional inspection layer improves the gateway's ability to identify suspicious or unwanted email.

Figure 7.3 Rspamd Service Verification

```
ir@GatewayVM:~$ sudo systemctl status rspamd
● rspamd.service - rapid spam filtering system
   Loaded: loaded (/lib/systemd/system/rspamd.service; enabled; ven>
   Active: active (running) since Fri 2026-06-26 18:00:26 UTC; 2h 4>
     Docs: https://rspamd.com/doc/
   Main PID: 1048 (rspamd)
     Tasks: 5 (limit: 3415)
    Memory: 121.5M
       CPU: 2.306s
    CGroup: /system.slice/rspamd.service
            └─1048 "rspamd: main process; 2.0 msg/sec, 0.0 msg/sec s>
              └─1731 "rspamd: rspamd_proxy process (localhost:11332)"
                └─1732 "rspamd: controller process (localhost:11334)" ""
                  └─1733 "rspamd: normal process (localhost:11333)" "" "" >
                    └─1734 "rspamd: hs_helper process" "" "" "" "" "" "" "" >

Jun 26 18:00:26 GatewayVM systemd[1]: Started rapid spam filtering sy>
ir@GatewayVM:~$ sudo rspamadm configtest
syntax OK
ir@GatewayVM:~$
```

7.6 SMTP Monitoring Using Fail2Ban

Fail2Ban was configured to monitor Postfix log files for suspicious SMTP-related activities.

Instead of permanently blocking users, Fail2Ban automatically identifies repeated abnormal behaviour and temporarily restricts further access according to the configured policy.

During implementation, the Postfix jail was configured with the following parameters:

Parameter	Value
Jail	postfix
Maximum Retry	3

Find Time	600 Seconds
Ban Time	3600 Seconds

These settings provide a basic level of protection against repeated suspicious SMTP events.

Figure 7.4 Fail2Ban Configuration

```
lr@GatewayVM:~$ sudo fail2ban-client status
Status
|- Number of jail:      2
`- Jail list: postfix, sshd
lr@GatewayVM:~$ sudo fail2ban-client status postfix
Status for the jail: postfix
|- Filter
|  |- Currently failed: 0
|  |- Total failed:    0
|  `-- File list:      /var/log/mail.log
`- Actions
   |- Currently banned: 0
   |- Total banned:    0
   `-- Banned IP list:
lr@GatewayVM:~$
```

7.7 Layered Security Model

The implemented Mail Security Gateway follows a defense-in-depth approach by combining multiple independent security controls.

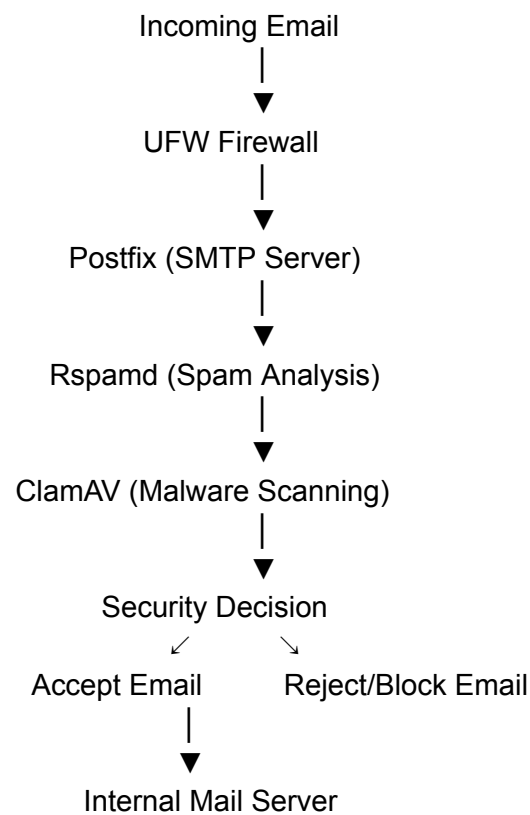
Each component performs a specific task while supporting the overall security of the email infrastructure.

Security Component	Primary Function
UFW	Restricts network access
Postfix	Handles SMTP communication
Rspamd	Performs spam analysis

ClamAV	Detects malware
Fail2Ban	Monitors suspicious SMTP activity

Rather than depending on a single security mechanism, the gateway uses multiple protective layers to reduce the likelihood of malicious email reaching the Internal Mail Server.

Figure 7.5 Layered Security Architecture



7.8 Security Verification

After completing the hardening process, all implemented security controls were verified.

The verification confirmed:

- The firewall rules were active.
- Mail services were operational.
- Rspamd was successfully analyzing email.
- ClamAV was scanning attachments.
- Fail2Ban was monitoring SMTP events.

These verification steps ensured that the gateway was fully prepared for the validation scenarios presented in the following chapters.

Figure 7.6 Security Verification

```

ir@GatewayVM:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
25/tcp ALLOW IN 192.168.56.0/24
22/tcp (v6) ALLOW IN Anywhere (v6)
25/tcp (v6) ALLOW IN Anywhere (v6)

ir@GatewayVM:~$ sudo systemctl status postfix
● postfix.service - Postfix Mail Transport Agent
   Loaded: loaded (/lib/systemd/system/postfix.service; enabled; ve>
   Active: active (exited) since Fri 2026-06-26 18:00:39 UTC; 1h 51>
   Docs: man:postfix(1)
   Process: 1700 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
   Process: 2847 ExecReload=/bin/true (code=exited, status=0/SUCCESS)
   Main PID: 1700 (code=exited, status=0/SUCCESS)
   CPU: 1ms

Jun 26 18:00:39 GatewayVM systemd[1]: Starting Postfix Mail Transport>
Jun 26 18:00:39 GatewayVM systemd[1]: Finished Postfix Mail Transport>
Jun 26 18:02:38 GatewayVM systemd[1]: Reloading Postfix Mail Transpor>
Jun 26 18:02:38 GatewayVM systemd[1]: Reloaded Postfix Mail Transport>
ir@GatewayVM:~$ sudo systemctl status rspamd
● rspamd.service - rapid spam filtering system
   Loaded: loaded (/lib/systemd/system/rspamd.service; enabled; ven>
   Active: active (running) since Fri 2026-06-26 18:00:26 UTC; 1h 5>
   Docs: https://rspamd.com/doc/
   Main PID: 1048 (rspamd)
   Tasks: 5 (limit: 3415)
   Memory: 123.8M
   CPU: 2.169s
   CGroup: /system.slice/rspamd.service
           └─1048 "rspamd: main process; 2.0 msg/sec, 0.0 msg/sec s>
             └─1731 "rspamd: rspamd_proxy process (localhost:11332)"
             └─1732 "rspamd: controller process (localhost:11334)" ""
             └─1733 "rspamd: normal process (localhost:11333)" "" "" >
             └─1734 "rspamd: hs_helper process" "" "" "" "" "" "" "" >

Jun 26 18:00:26 GatewayVM systemd[1]: Started rapid spam filtering sy>

```



```

lr@GatewayVM:~$ sudo systemctl status clamav-daemon
● clamav-daemon.service - Clam AntiVirus userspace daemon
   Loaded: loaded (/lib/systemd/system/clamav-daemon.service; enabled; vendor preset: enabled)
   Drop-In: /etc/systemd/system/clamav-daemon.service.d
            └─extend.conf
   Active: active (running) since Fri 2026-06-26 18:00:15 UTC; 1h 5m; 1s ago
   TriggeredBy: ● clamav-daemon.socket
     Docs: man:clamd(8)
           man:clamd.conf(5)
           https://docs.clamav.net/
   Process: 790 ExecStartPre=/bin/mkdir -p /run/clamav (code=exited, status=0/SUCCESS)
   Process: 812 ExecStartPre=/bin/chown clamav /run/clamav (code=exited, status=0/SUCCESS)
  Main PID: 818 (clamd)
    Tasks: 2 (limit: 3415)
   Memory: 1.0G
      CPU: 11.620s
   CGroup: /system.slice/clamav-daemon.service
           └─818 /usr/sbin/clamd --foreground=true

Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> OLE
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> PDF
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> SWF
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> HTM
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> XML
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> HWP
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> One
lr@GatewayVM:~$ sudo systemctl status fail2ban
● fail2ban.service - Fail2Ban Service
   Loaded: loaded (/lib/systemd/system/fail2ban.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2026-06-26 18:00:19 UTC; 1h 5m; 1s ago
     Docs: man:fail2ban(1)
  Main PID: 911 (fail2ban-server)
    Tasks: 7 (limit: 3415)
   Memory: 15.4M
      CPU: 3.969s
   CGroup: /system.slice/fail2ban.service
           └─911 /usr/bin/python3 /usr/bin/fail2ban-server -xf start

Jun 26 18:00:19 GatewayVM systemd[1]: Started Fail2Ban Service.
Jun 26 18:00:27 GatewayVM fail2ban-server[911]: Server ready
lr@GatewayVM:~$

```

7.9 Chapter Summary

This chapter described the security hardening measures implemented on the Mail Security Gateway following its deployment. UFW was configured to restrict unnecessary network access, ClamAV was used to scan email attachments for malware, Rspamd was configured to inspect

incoming messages for spam, and Fail2Ban was deployed to monitor suspicious SMTP activity. Together, these components established a layered security architecture that strengthened the overall protection of the email infrastructure. With the security controls successfully implemented and verified, the system was ready for functional validation, beginning with legitimate email delivery in the next chapter.

Chapter 8

Validation of Legitimate Email Delivery

8.1 Introduction

After deploying and securing the Mail Security Gateway, the first validation was performed to verify that legitimate email messages could successfully pass through the gateway and reach the Internal Mail Server.

This validation is important because a secure email gateway must not only block malicious messages but also allow legitimate communication without unnecessary interruption. Before testing spam filtering and malware detection, it was essential to confirm that the basic mail delivery process operated correctly.

This chapter documents the procedure, observations, and results of the legitimate email validation.

8.2 Validation Objective

The objective of this validation was to confirm that:

- The client could establish an SMTP connection with the Mail Security Gateway.
- GatewayVM successfully received the email.
- The Mail Security Gateway inspected the message.

- The Internal Mail Server received the approved email.
- The recipient mailbox successfully stored the delivered message.

Successful completion of these steps would demonstrate that the email infrastructure was functioning correctly before introducing malicious test cases.

8.3 Test Environment

The validation was performed using the virtual laboratory developed during the implementation phase.

The participating systems were:

System	Role
Ubuntu Client	Email Sender
GatewayVM	Mail Security Gateway
ubuntu1	Internal Mail Server
mailuser	Recipient Mailbox

The email was generated from the Ubuntu Client and addressed to the **mailuser** account hosted on the Internal Mail Server.

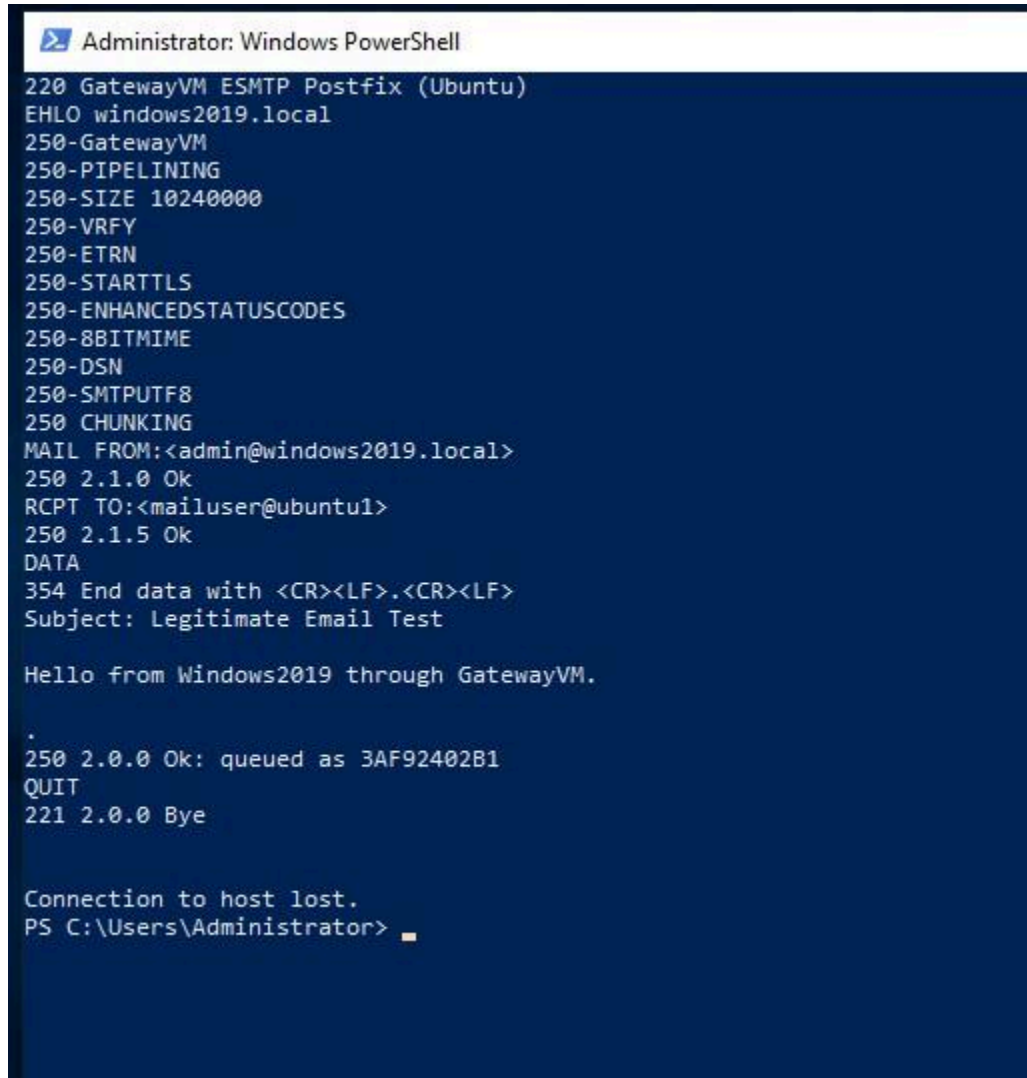
8.4 Validation Procedure

The following procedure was followed during testing:

1. A legitimate email message was created on the Ubuntu Client.
2. The message was submitted to GatewayVM using SMTP.
3. GatewayVM accepted the SMTP connection.
4. Postfix processed the message.
5. Rspamd inspected the email.
6. Since no malicious content was detected, the email was approved.
7. GatewayVM relayed the message to the Internal Mail Server.
8. The recipient mailbox was checked to verify successful delivery.

Each stage was monitored using system logs to confirm that the message followed the expected processing path.

Figure 8.1 Sending the Legitimate Email



```
Administrator: Windows PowerShell
220 GatewayVM ESMTP Postfix (Ubuntu)
EHLO windows2019.local
250-GatewayVM
250-PIPELINING
250-SIZE 10240000
250-VRFY
250-ETRN
250-STARTTLS
250-ENHANCEDSTATUSCODES
250-8BITMIME
250-DSN
250-SMTPUTF8
250 CHUNKING
MAIL FROM:<admin@windows2019.local>
250 2.1.0 Ok
RCPT TO:<mailuser@ubuntu1>
250 2.1.5 Ok
DATA
354 End data with <CR><LF>.<CR><LF>
Subject: Legitimate Email Test

Hello from Windows2019 through GatewayVM.

.
250 2.0.0 Ok: queued as 3AF92402B1
QUIT
221 2.0.0 Bye

Connection to host lost.
PS C:\Users\Administrator>
```

8.5 Gateway Processing

Once the email reached GatewayVM, the SMTP transaction was processed by Postfix and inspected by Rspamd.

The gateway confirmed that the email contained no suspicious content or malicious attachments. As a result, the message was accepted and forwarded to the Internal Mail Server.

This demonstrates that the security inspection process does not interfere with normal communication when no threats are detected.

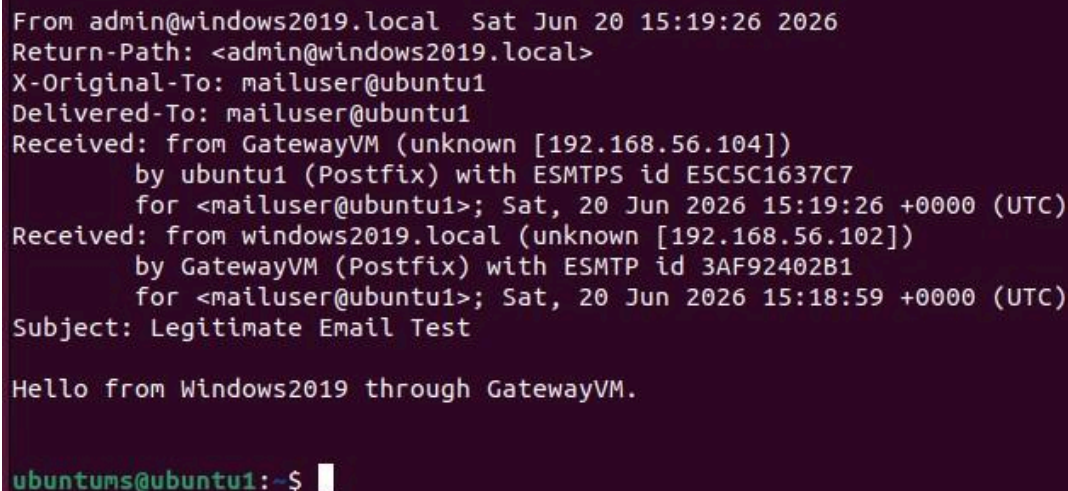
8.6 Delivery to the Internal Mail Server

After passing through the Mail Security Gateway, the message was successfully relayed to **ubuntu1**.

The recipient mailbox (**mailuser**) was examined, confirming that the email had been delivered correctly.

Successful delivery verified that the communication path between the gateway and the Internal Mail Server was functioning as expected.

Figure 8.3 Successful Email Delivery



```
From admin@windows2019.local Sat Jun 20 15:19:26 2026
Return-Path: <admin@windows2019.local>
X-Original-To: mailuser@ubuntu1
Delivered-To: mailuser@ubuntu1
Received: from GatewayVM (unknown [192.168.56.104])
    by ubuntu1 (Postfix) with ESMTPS id E5C5C1637C7
    for <mailuser@ubuntu1>; Sat, 20 Jun 2026 15:19:26 +0000 (UTC)
Received: from windows2019.local (unknown [192.168.56.102])
    by GatewayVM (Postfix) with ESMTTP id 3AF92402B1
    for <mailuser@ubuntu1>; Sat, 20 Jun 2026 15:18:59 +0000 (UTC)
Subject: Legitimate Email Test

Hello from Windows2019 through GatewayVM.

ubuntu1@ubuntu1:~$
```

8.7 Log Verification

System logs were reviewed to verify each stage of the SMTP transaction.

The logs confirmed:

- SMTP connection establishment.
- Message acceptance by GatewayVM.
- Successful relay to the Internal Mail Server.
- Final delivery to the recipient mailbox.

Log analysis confirmed that the message followed the expected processing workflow without errors.

8.8 Validation Results

The results of the legitimate email validation are summarized below.

Validation Item	Result
SMTP connection established	✓ Passed
Gateway accepted the email	✓ Passed
Email successfully inspected	✓ Passed
Email relayed to Internal Mail Server	✓ Passed
Recipient mailbox received the email	✓ Passed

The successful completion of all validation steps confirmed that the Mail Security Gateway was capable of handling legitimate email without disrupting normal communication.

8.9 Discussion

This validation demonstrated that the Mail Security Gateway successfully processed and delivered legitimate email while maintaining the expected security workflow.

Although the gateway performed spam inspection before forwarding the message, the validation confirmed that normal communication remained unaffected when no security threats were present.

Establishing this baseline was essential before evaluating the system's response to malicious email in the following chapter.

8.10 Chapter Summary

This chapter validated the successful delivery of legitimate email through the implemented Mail Security Gateway. A test message was transmitted from the Ubuntu Client, inspected by GatewayVM, and successfully delivered to the Internal Mail Server. Log analysis and mailbox verification confirmed that the email infrastructure functioned correctly, establishing a reliable baseline for the malware detection validation presented in the next chapter.

Chapter 9

Validation of Malware Detection and Prevention

9.1 Introduction

After confirming that legitimate email could successfully pass through the Mail Security Gateway, the next stage of validation focused on malware detection and prevention.

One of the primary objectives of the Mail Security Gateway is to prevent malicious email attachments from reaching users. To verify this capability, a controlled malware simulation was performed using the **EICAR Anti-Malware Test File**. The EICAR file is an industry-standard antivirus test file that safely simulates malware without causing harm to the operating system.

This chapter describes the validation process and demonstrates how the implemented Mail Security Gateway successfully detected and rejected a malicious email before it reached the Internal Mail Server.

9.2 Validation Objective

The objectives of this validation were:

- Verify that the Mail Security Gateway inspects email attachments.
- Confirm that ClamAV detects malicious files.
- Verify communication between ClamAV and Rspamd.
- Confirm that Postfix rejects infected email.
- Ensure that malicious attachments are prevented from reaching the recipient mailbox.

Successful completion of these objectives would demonstrate the effectiveness of the layered email security architecture.

9.3 Test Environment

The malware validation was performed using the same virtual laboratory used throughout the project.

The participating systems were:

System	Role
Ubuntu Client	Email Sender
GatewayVM	Mail Security Gateway
ClamAV	Malware Detection Engine
Rspamd	Email Analysis Engine
ubuntu1	Internal Mail Server

The EICAR test file was attached to a legitimate email message generated from the Ubuntu Client.

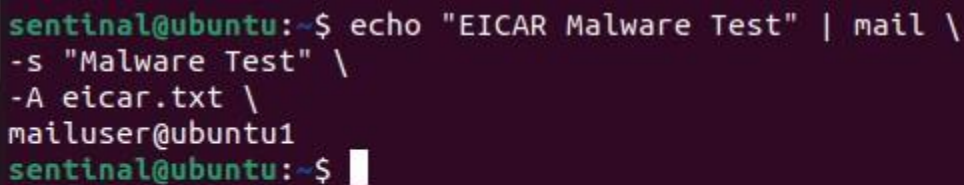
9.4 Test Procedure

The following procedure was followed during validation:

1. The EICAR antivirus test file was created on the Ubuntu Client.
2. A new email message was prepared with the EICAR file attached.
3. The message was sent to the Mail Security Gateway.
4. GatewayVM accepted the SMTP connection.
5. Rspamd analyzed the email.
6. ClamAV scanned the attachment.
7. The EICAR signature was detected.
8. Rspamd instructed Postfix to reject the message.
9. The email was prevented from reaching the Internal Mail Server.

Throughout the validation, system logs were monitored to verify every stage of the inspection process.

Figure 9.1 Preparing the Malware Test



```
sentinal@ubuntu:~$ echo "EICAR Malware Test" | mail \
-s "Malware Test" \
-A eicar.txt \
mailuser@ubuntu1
sentinal@ubuntu:~$
```

9.5 Malware Detection Process

After the email reached GatewayVM, Rspamd began processing the message and submitted the attachment to ClamAV for antivirus scanning.

ClamAV successfully identified the attachment as the **EICAR test signature**, confirming that the antivirus engine was functioning correctly.

Following detection, ClamAV reported the result to Rspamd, which immediately instructed Postfix to reject the SMTP transaction.

This layered interaction between Postfix, Rspamd, and ClamAV demonstrates how multiple security components work together to protect the Internal Mail Server.

Figure 9.2 ClamAV Detecting the EICAR Signature

```
2026-06-23 08:16:13 #1773(normal) <fb2e09>; lua; common.lua:108: clamav: result - virusfound: "Eicar-Signature - score: 1"
2026-06-23 08:16:13 #1773(normal) <fb2e09>; task; rspamd_add_passthrough_result: <20260623081613.C3F8D424FB@GatewayVM>; set pre-
result to 'reject' (no score): 'clamav: virus found: "Eicar-Signature"' from clamav(1)
2026-06-23 08:16:14 #1773(normal) <fb2e09>; task; rspamd_task_write_log: id: <20260623081613.C3F8D424FB@GatewayVM>, qid: <C3F8D4
24FB>, ip: 127.0.0.1, from: <ir@gateway.lab>, (default: T (reject): [1.15/15.00] [SUBJ_ALL_CAPS(0.75){10;},MID_RHS_NOT_FQDN(0.50
){},MIME_GOOD(-0.10){multipart/mixed;text/plain;},ARC_NA(0.00){},CLAM_VIRUS(0.00){Eicar-Signature;},FROM_EQ_ENVFROM(0.00){},FROM
_HAS_DN(0.00){},HAS_ATTACHMENT(0.00){},MIME_TRACE(0.00){0:+:1:+:2:~:},RCPT_COUNT_ONE(0.00){1;},RCVD_COUNT_ZERO(0.00){0;},TO_DN_N
ONE(0.00){},TO_MATCH_ENVRCPT_ALL(0.00){}]), len: 1093, time: 242.887ms, dns req: 0, digest: <8d19947698d83a64e9cd2c3417a87f5e>,
rcpts: <mailuser@ubuntu1>, mime_rcpts: <mailuser@ubuntu1>, forced: reject "clamav: virus found: "Eicar-Signature""; score=nan (s
et by clamav)
```

9.6 Email Rejection

After receiving the malware detection result, Postfix terminated the SMTP transaction and rejected the email.

Because the rejection occurred at the gateway, the malicious message was never forwarded to the Internal Mail Server.

This behaviour confirms that the Mail Security Gateway successfully enforced its security policy before delivery.

Figure 9.3 Gateway Rejecting the Email

```
Jun 24 09:54:51 gateway postfix/smtpd[3908]: NOQUEUE: reject: RCPT fro
m unknown[192.168.56.1]: 454 4.7.1 <mailuser@ubuntu1>: Relay access de
nied; from=<sentinal@ubuntu> to=<mailuser@ubuntu1> proto=ESMTP helo=<u
buntu>
Jun 24 09:54:51 gateway postfix/smtpd[3905]: NOQUEUE: reject: RCPT fro
m unknown[192.168.56.1]: 454 4.7.1 <mailuser@ubuntu1>: Relay access de
nied; from=<sentinal@ubuntu> to=<mailuser@ubuntu1> proto=ESMTP helo=<u
buntu>
Jun 26 18:39:15 gateway postfix/cleanup[3102]: A2E03402AF: milter-reje
ct: END-OF-MESSAGE from unknown[192.168.56.101]: 5.7.1 clamav: virus f
ound: "Eicar-Signature"; from=<sentinal@ubuntu> to=<mailuser@ubuntu1>
proto=ESMTP helo=<ubuntu>
```

9.7 Log Analysis

The system logs were examined to verify each stage of the malware detection process.

The logs confirmed:

- Successful SMTP connection.
- Attachment analysis.
- Detection of the EICAR signature.
- Malware identification by ClamAV.
- Rspamd security decision.
- Postfix rejection.
- No delivery to the Internal Mail Server.

The log entries provided clear evidence that the security components operated as expected throughout the validation.

Figure 9.4 Gateway Log Verification

```
m unknown[192.168.56.1]: 454 4.7.1 <mailuser@ubuntu1>: Relay access de
nied; from=<sentinal@ubuntu> to=<mailuser@ubuntu1> proto=ESMTP helo=<u
buntu>
Jun 24 09:54:51 gateway postfix/smtpd[3904]: NOQUEUE: reject: RCPT fro
m unknown[192.168.56.1]: 454 4.7.1 <mailuser@ubuntu1>: Relay access de
nied; from=<sentinal@ubuntu> to=<mailuser@ubuntu1> proto=ESMTP helo=<u
buntu>
Jun 24 09:54:51 gateway postfix/smtpd[3906]: NOQUEUE: reject: RCPT fro
m unknown[192.168.56.1]: 454 4.7.1 <mailuser@ubuntu1>: Relay access de
nied; from=<sentinal@ubuntu> to=<mailuser@ubuntu1> proto=ESMTP helo=<u
buntu>
Jun 24 09:54:51 gateway postfix/smtpd[3908]: NOQUEUE: reject: RCPT fro
m unknown[192.168.56.1]: 454 4.7.1 <mailuser@ubuntu1>: Relay access de
nied; from=<sentinal@ubuntu> to=<mailuser@ubuntu1> proto=ESMTP helo=<u
buntu>
Jun 24 09:54:51 gateway postfix/smtpd[3905]: NOQUEUE: reject: RCPT fro
m unknown[192.168.56.1]: 454 4.7.1 <mailuser@ubuntu1>: Relay access de
nied; from=<sentinal@ubuntu> to=<mailuser@ubuntu1> proto=ESMTP helo=<u
buntu>
Jun 26 18:39:15 gateway postfix/cleanup[3102]: A2E03402AF: milter-reje
ct: END-OF-MESSAGE from unknown[192.168.56.101]: 5.7.1 clamav: virus f
ound: "Eicar-Signature"; from=<sentinal@ubuntu> to=<mailuser@ubuntu1>
proto=ESMTP helo=<ubuntu>
lr@GatewayVM:~$
```

9.8 Validation Results

The results of the malware validation are summarized below.

Validation Item	Result
Attachment inspected	✓ Passed
ClamAV detected the EICAR signature	✓ Passed
Rspamd received the detection result	✓ Passed
Gateway rejected the email	✓ Passed
Internal Mail Server protected	✓ Passed

The successful completion of these validation steps demonstrates that the Mail Security Gateway effectively prevented malicious email from reaching the recipient.

9.9 Discussion

This validation confirmed that the integration between Postfix, Rspamd, and ClamAV operated correctly within the implemented architecture.

Instead of relying on a single security mechanism, the gateway combined multiple services to inspect the email, scan its attachment, evaluate the security result, and enforce the appropriate action.

The use of the EICAR test file provided a safe and widely accepted method for verifying antivirus functionality without introducing actual malware into the laboratory environment.

9.10 Key Findings

The malware validation demonstrated several important characteristics of the implemented Mail Security Gateway:

- Every email attachment was inspected before delivery.
- ClamAV successfully detected the EICAR antivirus test signature.
- Rspamd processed the antivirus result and coordinated the security decision.
- Postfix rejected the infected email before it reached the Internal Mail Server.
- The layered security architecture functioned exactly as intended.

These observations confirm that the gateway provides effective protection against malicious email attachments within the scope of the laboratory environment.

9.11 Chapter Summary

This chapter validated the malware detection capabilities of the implemented Mail Security Gateway. Using the EICAR antivirus test file, the system successfully demonstrated attachment inspection, malware detection, policy enforcement, and email rejection before delivery. The validation confirmed that the integration between Postfix, Rspamd, and ClamAV effectively protected the Internal Mail Server from malicious email attachments while maintaining the layered security design introduced earlier in the report.

Chapter 10

Validation of SMTP Abuse Resistance and Connection Management

10.1 Introduction

Following the successful validation of legitimate email delivery and malware detection, the final validation focused on the system's behaviour during abnormal SMTP activity.

A Mail Security Gateway should not only detect malicious email attachments but also remain stable when exposed to excessive or repeated SMTP connections. To evaluate this capability, controlled SMTP connection tests were performed using the Ubuntu Client and Windows Server 2019.

The objective was to observe how the implemented environment responded when multiple email messages were transmitted within a short period.

10.2 Validation Objective

The objectives of this validation were:

- Verify that GatewayVM continued operating during repeated SMTP activity.
 - Observe the behaviour of the Internal Mail Server under increased SMTP connections.
 - Confirm that the gateway continued processing SMTP requests correctly.
 - Validate that temporary connection limits were enforced.
 - Verify that email processing remained stable throughout the test.
-

10.3 Test Environment

The validation used the same laboratory environment developed during the implementation phase.

The participating systems were:

System	Role
Ubuntu Client	SMTP Traffic Generator
Windows Server 2019	Secondary SMTP Traffic Generator
GatewayVM	Mail Security Gateway
ubuntu1	Internal Mail Server

Both Linux and Windows clients were used to generate multiple SMTP requests, allowing the behaviour of the gateway and mail server to be observed from different client systems.

10.4 Test Procedure

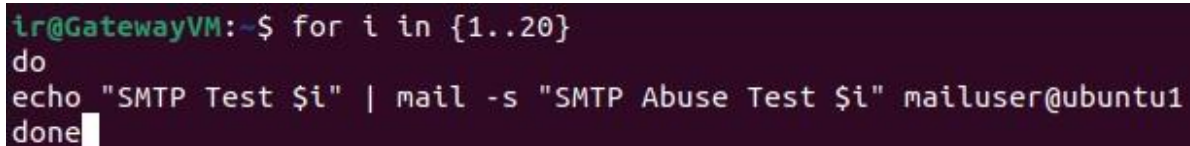
The validation was performed using a controlled sequence of repeated email transmissions.

The procedure consisted of the following steps:

1. Multiple email messages were generated from the client systems.
2. GatewayVM received the SMTP requests.
3. Postfix processed each SMTP transaction.
4. GatewayVM attempted to relay approved messages to the Internal Mail Server.
5. The Internal Mail Server monitored the number of simultaneous SMTP connections.
6. System logs were continuously observed throughout the test.

The objective was not to perform a denial-of-service attack but to evaluate the behaviour of the implemented environment during increased SMTP activity.

Figure 10.1 SMTP Abuse Test



```
ir@GatewayVM:~$ for i in {1..20}
do
echo "SMTP Test $i" | mail -s "SMTP Abuse Test $i" mailuser@ubuntu1
done
```

10.5 Observed System Behaviour

During the test, GatewayVM successfully accepted incoming SMTP connections and processed each email according to the configured workflow.

As the number of concurrent relay attempts increased, the Internal Mail Server began enforcing its SMTP connection limits.

Instead of accepting an unlimited number of simultaneous connections, the server temporarily refused additional SMTP sessions until existing connections were released.

This behaviour demonstrated that the mail infrastructure remained stable while protecting itself from excessive connection requests.

Figure 10.2 Connection Limit Reached

```

Jun  8 08:18:12 ubuntu1 postfix/smtpd[6617]: connect from unknown[192.168.56.101]
Jun  8 08:18:14 ubuntu1 postfix/smtpd[6617]: lost connection after CONNECT from unknown[192.168.56.101]
Jun  8 08:18:14 ubuntu1 postfix/smtpd[6617]: disconnect from unknown[192.168.56.101] commands=0 /0
Jun  8 08:18:14 ubuntu1 postfix/smtpd[6617]: connect from unknown[192.168.56.101]
Jun  8 08:18:16 ubuntu1 postfix/smtpd[6617]: lost connection after CONNECT from unknown[192.168.56.101]
Jun  8 08:18:16 ubuntu1 postfix/smtpd[6617]: disconnect from unknown[192.168.56.101] commands=0 /0
Jun  8 08:18:16 ubuntu1 postfix/smtpd[6617]: connect from unknown[192.168.56.101]
Jun  8 08:18:16 ubuntu1 postfix/smtpd[6617]: warning: Connection rate limit exceeded: 11 from unknown[192.168.56.101] for service smtp
Jun  8 08:18:16 ubuntu1 postfix/smtpd[6617]: disconnect from unknown[192.168.56.101] commands=0 /0
Jun  8 08:18:16 ubuntu1 postfix/smtpd[6617]: connect from unknown[192.168.56.101]
Jun  8 08:18:16 ubuntu1 postfix/smtpd[6617]: warning: Connection rate limit exceeded: 12 from unknown[192.168.56.101] for service smtp
Jun  8 08:18:16 ubuntu1 postfix/smtpd[6617]: disconnect from unknown[192.168.56.101] commands=0 /0
Jun  8 08:18:16 ubuntu1 postfix/smtpd[6617]: connect from unknown[192.168.56.101]
Jun  8 08:18:16 ubuntu1 postfix/smtpd[6617]: warning: Connection rate limit exceeded: 13 from unknown[192.168.56.101] for service smtp
Jun  8 08:18:16 ubuntu1 postfix/smtpd[6617]: disconnect from unknown[192.168.56.101] commands=0 /0
Jun  8 08:18:16 ubuntu1 postfix/smtpd[6617]: connect from unknown[192.168.56.101]
Jun  8 08:18:16 ubuntu1 postfix/smtpd[6617]: warning: Connection rate limit exceeded: 14 from unknown[192.168.56.101] for service smtp
Jun  8 08:18:16 ubuntu1 postfix/smtpd[6617]: disconnect from unknown[192.168.56.101] commands=0 /0
Jun  8 08:18:16 ubuntu1 postfix/smtpd[6617]: connect from unknown[192.168.56.101]
Jun  8 08:18:16 ubuntu1 postfix/smtpd[6617]: warning: Connection rate limit exceeded: 15 from unknown[192.168.56.101] for service smtp
Jun  8 08:18:16 ubuntu1 postfix/smtpd[6617]: disconnect from unknown[192.168.56.101] commands=0 /0
ubuntu1@ubuntu1:~$ postconf | grep smtpd_client_connection
postscreen_client_connection_count_limit = $smtpd_client_connection_count_limit
smtpd_client_connection_count_limit = 5
smtpd_client_connection_rate_limit = 10
smtpd_client_event_limit_exceptions = ${smtpd_client_connection_limit_exceptions:$mynetworks}
ubuntu1@ubuntu1:~$

```

10.6 Gateway Response

During the SMTP validation, GatewayVM continued to receive and process incoming SMTP requests while enforcing the configured security policies. When the client exceeded the configured SMTP connection rate, Postfix generated warning messages indicating that the connection rate limit had been exceeded. Despite these warnings, the gateway services remained operational and continued processing legitimate SMTP traffic. This behaviour

demonstrates that the configured connection control mechanism can identify excessive connection attempts while maintaining the stability of the mail gateway.

Figure 10.3 Gateway Service Status During SMTP Validation

```
ir@GatewayVM:~$ sudo systemctl status postfix
● postfix.service - Postfix Mail Transport Agent
   Loaded: loaded (/lib/systemd/system/postfix.service; enabled; ve>
   Active: active (exited) since Fri 2026-06-26 18:00:39 UTC; 1h 10>
   Docs: man:postfix(1)
   Process: 1700 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
   Process: 2847 ExecReload=/bin/true (code=exited, status=0/SUCCESS)
  Main PID: 1700 (code=exited, status=0/SUCCESS)
     CPU: 1ms

Jun 26 18:00:39 GatewayVM systemd[1]: Starting Postfix Mail Transport>
Jun 26 18:00:39 GatewayVM systemd[1]: Finished Postfix Mail Transport>
Jun 26 18:02:38 GatewayVM systemd[1]: Reloading Postfix Mail Transpor>
Jun 26 18:02:38 GatewayVM systemd[1]: Reloaded Postfix Mail Transport>
lines 1-13/13 (END)
```

10.7 Log Analysis

GatewayVM logs were examined throughout the validation.

The logs confirmed:

- Successful SMTP connection establishment.
- Message processing by Postfix.
- Relay attempts to the Internal Mail Server.
- Temporary SMTP connection limitation.
- Deferred delivery handling.

The repeated occurrence of the following response confirmed that the Internal Mail Server was enforcing connection management:

421 4.7.0 Error: too many connections

The gateway itself remained operational throughout the validation.

Figure 10.4 Gateway Log Verification

```
Jun 26 19:13:44 gateway postfix/smtp[9242]: 9A3A642440: to=<mailuser@u
buntu1>, relay=192.168.56.103[192.168.56.103]:25, delay=0.58, delays=0
.27/0.26/0.05/0, dsn=4.7.0, status=deferred (host 192.168.56.103[192.1
68.56.103] refused to talk to me: 421 4.7.0 ubuntu1 Error: too many co
nnections from 192.168.56.104)
ir@GatewayVM:~$
```

10.8 Validation Results

The results of the SMTP validation are summarized below.

Validation Item	Result
Gateway accepted SMTP connections	✓ Passed
Mail processing continued	✓ Passed
Internal Mail Server enforced connection limits	✓ Passed
Deferred delivery handled correctly	✓ Passed
Gateway remained operational	✓ Passed

The validation confirmed that the implemented architecture remained stable under increased SMTP activity.

10.9 Discussion

This validation demonstrated that the Mail Security Gateway continued processing email normally even when the Internal Mail Server temporarily limited the number of simultaneous SMTP connections.

Rather than indicating a system failure, the **421 4.7.0** response represents a protective mechanism designed to prevent resource exhaustion during periods of increased SMTP activity.

The behaviour observed during testing illustrates the importance of connection management in maintaining the stability of an email infrastructure.

Although this validation was conducted within a laboratory environment, it demonstrates that the gateway and mail server interact correctly when temporary connection limits are reached.

10.10 Key Findings

The SMTP validation demonstrated the following:

- GatewayVM remained operational throughout the test.
- Postfix continued processing SMTP requests.
- The Internal Mail Server successfully enforced connection limits.
- Deferred delivery prevented unnecessary message loss.
- The implemented architecture remained stable under controlled SMTP load.

These findings confirm that the implemented Mail Security Gateway not only protects against malicious email but also operates reliably during periods of increased SMTP activity.

10.11 Chapter Summary

This chapter presented the final validation of the implemented Mail Security Gateway by evaluating its behaviour during controlled SMTP abuse testing. The gateway successfully processed repeated SMTP requests while the Internal Mail Server enforced temporary connection limits to protect system resources. GatewayVM remained fully operational, and Postfix correctly deferred message delivery when required. The successful completion of this validation demonstrates that the implemented architecture maintains stable and reliable email processing under increased SMTP activity within the laboratory environment.

Chapter 11

Troubleshooting and Problem Resolution

11.1 Introduction

During the implementation of the Mail Security Gateway, several technical challenges were encountered. These issues affected different stages of the deployment, including service configuration, communication between virtual machines, email delivery, malware detection, and system validation.

Rather than indicating implementation failure, these challenges formed an important part of the learning process. Each issue was analyzed, investigated, and resolved through systematic troubleshooting and configuration adjustments.

This chapter documents the most significant problems encountered during the project and explains the solutions that were implemented to achieve a fully functional Mail Security Gateway.

11.2 Troubleshooting Methodology

Whenever an issue occurred, a structured troubleshooting approach was followed instead of making random configuration changes.

The general troubleshooting process consisted of:

1. Identifying the observed problem.
2. Reviewing system logs.
3. Verifying service status.
4. Examining configuration files.
5. Applying corrective changes.
6. Restarting affected services.
7. Performing validation tests.

This systematic approach minimized unnecessary configuration changes and helped isolate the root cause of each problem.

11.3 Problem 1 – SMTP Relay Configuration

Problem

During the initial deployment, emails sent from the client systems were not successfully forwarded to the Internal Mail Server.

Cause

The SMTP relay configuration on GatewayVM was incomplete, preventing Postfix from forwarding messages correctly.

Solution

The Postfix relay configuration was reviewed and updated to correctly identify the Internal Mail Server as the relay destination. After restarting the Postfix service, successful mail forwarding was verified.

Result

Legitimate email was successfully relayed from GatewayVM to the Internal Mail Server.

Figure 11.1 SMTP Relay Configuration

Insert Screenshot

Filename

agent-registered.jpg

11.4 Problem 2 – ClamAV Integration

Problem

Initially, email attachments were not being scanned for malware.

Cause

The antivirus service was not fully integrated with the email processing workflow.

Solution

The ClamAV service was configured correctly and integrated with Rspamd. Service status and logs were verified to confirm successful communication between both components.

Result

Attachments were successfully scanned before email delivery.

Figure 11.2 ClamAV Verification

```
● clamav-daemon.service - Clam AntiVirus userspace daemon
   Loaded: loaded (/lib/systemd/system/clamav-daemon.service; enabled; vendor preset: enabled)
   Drop-In: /etc/systemd/system/clamav-daemon.service.d
            └─extend.conf
   Active: active (running) since Fri 2026-06-26 18:00:15 UTC; 1h 1min ago
   TriggeredBy: ● clamav-daemon.socket
     Docs: man:clamd(8)
           man:clamd.conf(5)
           https://docs.clamav.net/
   Process: 790 ExecStartPre=/bin/mkdir -p /run/clamav (code=exited, status=0/SUCCESS)
   Process: 812 ExecStartPre=/bin/chown clamav /run/clamav (code=exited, status=0/SUCCESS)
  Main PID: 818 (clamd)
    Tasks: 2 (limit: 3415)
   Memory: 1.0G
      CPU: 11.620s
   CGroup: /system.slice/clamav-daemon.service
           └─818 /usr/sbin/clamd --foreground=true

Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> Mail
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> OLE
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> PDF
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> SWF
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> HTML
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> XML
Jun 26 18:00:59 GatewayVM clamd[818]: Fri Jun 26 18:00:59 2026 -> HWP
lines 1-25
```

11.5 Problem 3 – Malware Validation

Problem

The first malware validation attempts did not produce the expected behaviour.

Cause

The testing procedure and antivirus integration required verification before the EICAR test file could be detected correctly.

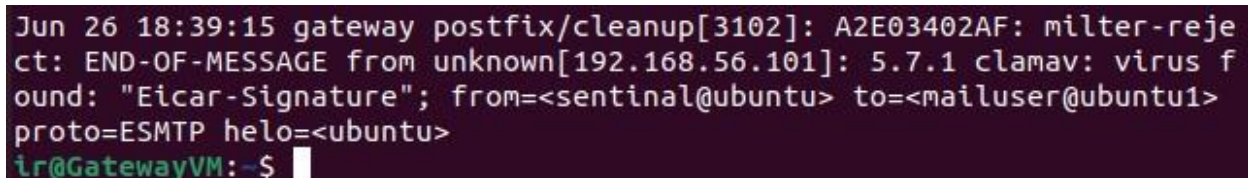
Solution

The EICAR test file was recreated and the ClamAV integration was verified using service logs. After confirming correct operation, the validation was repeated.

Result

The Mail Security Gateway successfully detected the EICAR test file and rejected the infected email before delivery.

Figure 11.3 Malware Detection

A terminal window with a dark background and light-colored text. The text shows a log entry from a gateway postfix cleanup process. It indicates that an email message was rejected because it contained a virus signature (EICAR). The log includes details like the message ID (A2E03402AF), the sender (unknown[192.168.56.101]), the virus detected (clamav: virus found: "Eicar-Signature"), and the email headers (from, to, proto, helo). The prompt at the bottom shows the user is at the GatewayVM.

```
Jun 26 18:39:15 gateway postfix/cleanup[3102]: A2E03402AF: milter-reject: END-OF-MESSAGE from unknown[192.168.56.101]: 5.7.1 clamav: virus found: "Eicar-Signature"; from=<sentinal@ubuntu> to=<mailuser@ubuntu1> proto=ESMTP helo=<ubuntu>  
ir@GatewayVM:~$
```

11.6 Problem 4 – SMTP Abuse Validation

Problem

The original objective was to observe the gateway's behaviour during repeated SMTP activity.

Cause

Several testing approaches were evaluated before identifying behaviour that accurately reflected the implemented system.

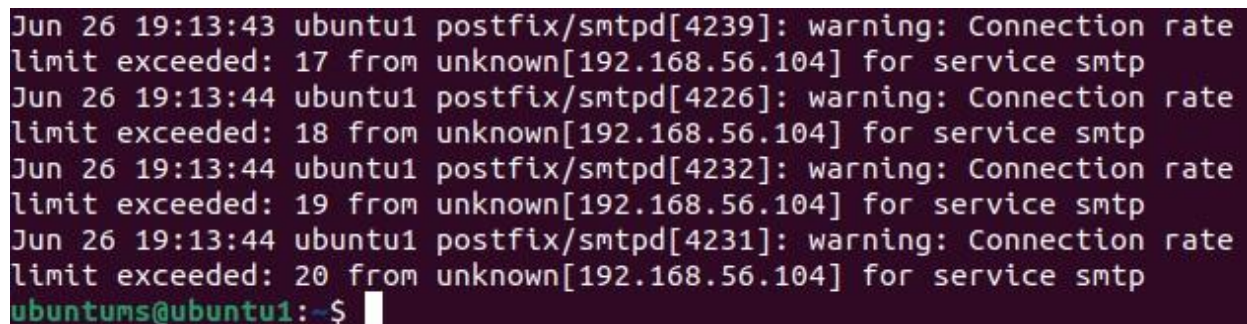
Solution

Controlled SMTP traffic was generated from both the Ubuntu Client and Windows Server 2019. System logs were monitored throughout the test to observe how the infrastructure responded to increased connection activity.

Result

The Internal Mail Server enforced temporary SMTP connection limits, while GatewayVM continued operating correctly and deferred affected messages for later delivery.

Figure 11.4 SMTP Validation

A terminal window screenshot with a dark background and light-colored text. It shows four lines of log messages from postfix/smtpd on an Ubuntu system. Each line indicates a warning that the connection rate limit has been exceeded for a specific service (smtp) from an unknown IP address (192.168.56.104). The connection rates are 17, 18, 19, and 20 respectively. The terminal prompt at the bottom is 'ubuntu@ubuntu1:~\$'.

```
Jun 26 19:13:43 ubuntu1 postfix/smtpd[4239]: warning: Connection rate
limit exceeded: 17 from unknown[192.168.56.104] for service smtp
Jun 26 19:13:44 ubuntu1 postfix/smtpd[4226]: warning: Connection rate
limit exceeded: 18 from unknown[192.168.56.104] for service smtp
Jun 26 19:13:44 ubuntu1 postfix/smtpd[4232]: warning: Connection rate
limit exceeded: 19 from unknown[192.168.56.104] for service smtp
Jun 26 19:13:44 ubuntu1 postfix/smtpd[4231]: warning: Connection rate
limit exceeded: 20 from unknown[192.168.56.104] for service smtp
ubuntu@ubuntu1:~$
```

11.7 Problem 5 – Network Communication

Problem

At different stages of the implementation, communication between virtual machines was interrupted.

Cause

Network configuration and virtual network settings required verification to ensure consistent communication between the systems.

Solution

The virtual network configuration was reviewed, IP addressing was verified, and connectivity tests were performed before continuing with the implementation.

Result

Stable communication was restored between all virtual machines.

Figure 11.5 Network Verification

```
sentinal@ubuntu:~$ ping -c 4 192.168.56.103
PING 192.168.56.103 (192.168.56.103) 56(84) bytes of data.
64 bytes from 192.168.56.103: icmp_seq=1 ttl=64 time=0.223 ms
64 bytes from 192.168.56.103: icmp_seq=2 ttl=64 time=0.288 ms
64 bytes from 192.168.56.103: icmp_seq=3 ttl=64 time=0.283 ms
64 bytes from 192.168.56.103: icmp_seq=4 ttl=64 time=0.285 ms

--- 192.168.56.103 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3119ms
rtt min/avg/max/mdev = 0.223/0.269/0.288/0.027 ms
sentinal@ubuntu:~$ ping -c 4 192.168.56.104
PING 192.168.56.104 (192.168.56.104) 56(84) bytes of data.
64 bytes from 192.168.56.104: icmp_seq=1 ttl=64 time=0.458 ms
64 bytes from 192.168.56.104: icmp_seq=2 ttl=64 time=0.262 ms
64 bytes from 192.168.56.104: icmp_seq=3 ttl=64 time=0.291 ms
64 bytes from 192.168.56.104: icmp_seq=4 ttl=64 time=0.285 ms

--- 192.168.56.104 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3175ms
rtt min/avg/max/mdev = 0.262/0.324/0.458/0.078 ms
sentinal@ubuntu:~$
```

11.8 Lessons Learned

Resolving these issues provided valuable practical experience beyond the initial implementation objectives.

Key lessons learned include:

- Careful planning simplifies system deployment.
- Reviewing logs is essential for identifying configuration problems.
- Security services should be validated individually before integration.
- Layered testing makes troubleshooting more efficient.
- Documentation is an important part of the implementation process.

These lessons significantly improved the quality of the final system and strengthened practical troubleshooting skills.

11.9 Summary of Issues and Resolutions

Issue	Resolution	Status
SMTP relay configuration	Updated Postfix relay settings	✓ Passed
ClamAV integration	Verified and integrated with Rspamd	✓ Passed
Malware validation	Revalidated using EICAR test file	✓ Passed
SMTP abuse validation	Observed connection management behaviour	✓ Passed
Network communication	Corrected network configuration	✓ Passed

The successful resolution of these issues resulted in a stable and fully functional Mail Security Gateway.

11.10 Chapter Summary

This chapter presented the major technical challenges encountered during the implementation of the Mail Security Gateway and described the methods used to resolve them. Through systematic troubleshooting, configuration verification, and repeated validation, each issue was successfully addressed. The experience gained during this process not only improved the final implementation but also strengthened practical skills in Linux administration, email security, and technical problem solving.

Chapter 12

Overall Project Evaluation

12.1 Introduction

After completing the implementation and validation phases, the overall performance of the Mail Security Gateway was evaluated. This evaluation measures how effectively the implemented system satisfied the objectives defined at the beginning of the project and identifies the strengths of the final solution.

The evaluation is based on the practical implementation, testing activities, and validation results presented in the previous chapters. Rather than focusing only on successful outcomes, this chapter also considers the practical limitations of the implemented laboratory environment.

12.2 Achievement of Project Objectives

At the beginning of the internship, several objectives were established to guide the implementation of the Mail Security Gateway. Upon completion of the project, each objective was reviewed to determine whether it had been successfully achieved.

The evaluation is summarized below.

Project Objective	Status
Deploy Internal Mail Server	✓ Achieved
Deploy Mail Security Gateway	✓ Achieved
Configure Secure SMTP Relay	✓ Achieved
Integrate Rspamd Spam Filtering	✓ Achieved
Integrate ClamAV Malware Detection	✓ Achieved
Configure UFW Firewall	✓ Achieved

Configure Fail2Ban Monitoring	✓ Achieved
Validate Legitimate Email Delivery	✓ Achieved
Validate Malware Detection	✓ Achieved
Evaluate SMTP Abuse Resistance	✓ Achieved

The results indicate that all primary project objectives were successfully completed within the scope of the internship.

12.3 Functional Evaluation

The implemented Mail Security Gateway performed successfully during all major functional tests.

The system demonstrated the ability to:

- Receive SMTP connections from client systems.
- Inspect incoming email before delivery.
- Detect malicious attachments using ClamAV.
- Analyze email using Rspamd.
- Relay legitimate email to the Internal Mail Server.
- Reject malicious email before delivery.
- Continue operating during periods of increased SMTP activity.

These results demonstrate that the core functionality of the Mail Security Gateway operated as expected throughout the validation process.

12.4 Security Evaluation

From a security perspective, the implemented solution provides multiple protective layers rather than relying on a single security mechanism.

The security architecture combines:

- Network protection through UFW.
- SMTP handling using Postfix.
- Spam analysis using Rspamd.

- Malware detection using ClamAV.
- Monitoring of suspicious SMTP activity using Fail2Ban.

This layered design improves the overall resilience of the email infrastructure by distributing security responsibilities across multiple independent components.

Although the implementation was performed in a laboratory environment, the architecture reflects common security principles used in modern email systems.

12.5 Validation Summary

Three practical validation scenarios were completed during the project.

Validation 1 – Legitimate Email Delivery

A normal email was successfully transmitted from the client system through GatewayVM and delivered to the Internal Mail Server.

Result: ✓ Successful

Validation 2 – Malware Detection

An email containing the EICAR antivirus test file was transmitted through the gateway.

ClamAV successfully detected the malicious signature, and the email was rejected before reaching the recipient.

Result: ✓ Successful

Validation 3 – SMTP Abuse Resistance

Multiple SMTP requests were generated within a short period to evaluate the system's behaviour during increased connection activity.

The Internal Mail Server enforced temporary connection limits, while GatewayVM continued operating correctly and deferred affected messages until communication could resume.

Result: ✓ Successful

12.6 Practical Skills Developed

Beyond the technical implementation, the internship provided significant practical experience in several areas of system administration and cybersecurity.

Skills developed during the project include:

- Linux server administration.
- Mail server deployment.
- SMTP configuration.
- Firewall management.
- Email security implementation.
- Malware detection.
- Service integration.
- System troubleshooting.
- Log analysis.
- Technical documentation.

These practical experiences strengthened the understanding of secure email infrastructure and demonstrated how multiple open-source technologies can be integrated into a functional security solution.

12.7 Project Strengths

The completed implementation demonstrates several important strengths.

- Layered security architecture.
- Separation of gateway and mail server responsibilities.
- Use of reliable open-source technologies.
- Successful malware detection.
- Stable SMTP communication.
- Effective email relay.
- Practical validation through multiple test scenarios.
- Comprehensive implementation documentation.

These strengths contribute to the reliability and educational value of the implemented system.

12.8 Overall Assessment

Based on the implementation results and validation activities, the project can be considered successfully completed.

The Mail Security Gateway achieved its intended purpose of protecting the Internal Mail Server by inspecting incoming email, detecting malicious attachments, and forwarding legitimate messages while maintaining stable communication between all participating systems.

Although developed within a controlled virtual environment, the project successfully demonstrates the design principles and implementation process of a secure email infrastructure using open-source technologies.

12.9 Chapter Summary

This chapter evaluated the completed Mail Security Gateway by reviewing the project objectives, functional capabilities, security features, and validation results. The evaluation confirmed that the implemented solution successfully achieved its primary objectives while providing practical experience in Linux system administration, email security, and cybersecurity implementation. The following chapter discusses the limitations of the current implementation and suggests possible improvements for future development.

Chapter 13

Project Limitations and Future Improvements

13.1 Introduction

Although the Mail Security Gateway was successfully designed, implemented, and validated, the completed system represents a laboratory-based implementation rather than a production deployment. Like any technical project, the implementation has several limitations that should be considered when evaluating the overall solution.

Identifying these limitations is an important part of the engineering process because it provides opportunities for future enhancement and demonstrates a realistic understanding of the implemented system.

This chapter discusses the major limitations encountered during the internship and proposes practical improvements that could further strengthen the Mail Security Gateway in future deployments.

13.2 Project Limitations

The implemented Mail Security Gateway successfully achieved its intended objectives within the scope of the internship. However, several limitations remain due to the controlled laboratory environment and the defined project scope.

13.2.1 Laboratory-Based Environment

The entire implementation was developed and tested using Oracle VirtualBox within an isolated Host-Only network.

Although this environment provided a safe platform for learning and experimentation, it does not fully represent the complexity of a production email infrastructure.

Real-world deployments involve public networks, multiple users, external DNS services, and significantly higher volumes of email traffic.

13.2.2 Limited Number of Client Systems

The validation was performed using two client systems:

- Ubuntu Client
- Windows Server 2019

While sufficient for demonstrating the functionality of the Mail Security Gateway, the environment does not simulate a large enterprise with hundreds or thousands of simultaneous users.

13.2.3 Limited Validation Scenarios

The project focused on three major validation scenarios:

- Legitimate email delivery.
- Malware detection using the EICAR test file.
- Controlled SMTP connection testing.

Although these scenarios successfully demonstrated the core functionality of the gateway, additional testing could be performed using larger datasets and more diverse attack simulations.

13.2.4 Basic Security Monitoring

The implemented solution includes monitoring through system logs and Fail2Ban.

However, the project does not include centralized log management, real-time alerting, or advanced security analytics typically found in enterprise Security Operations Centers (SOCs).

13.2.5 Performance Evaluation

The primary focus of this internship was functional implementation rather than performance benchmarking.

As a result, the project does not include measurements such as:

- Maximum email throughput.
- CPU utilization under heavy load.
- Memory consumption.
- Large-scale stress testing.

These measurements would be valuable in a production environment but were outside the scope of the internship.

13.3 Future Improvements

Although the current implementation successfully meets the project objectives, several enhancements could further improve the overall capability of the Mail Security Gateway.

13.3.1 High Availability

Future implementations could deploy multiple Mail Security Gateways operating together to provide redundancy and improve system availability.

This would reduce the impact of hardware or software failures and improve service continuity.

13.3.2 Secure Email Authentication

The security of the email infrastructure could be strengthened by implementing additional authentication mechanisms such as:

- SPF (Sender Policy Framework)
- DKIM (DomainKeys Identified Mail)
- DMARC (Domain-based Message Authentication, Reporting, and Conformance)

These technologies help reduce email spoofing and improve sender verification.

13.3.3 Enhanced Monitoring

Future implementations could integrate centralized monitoring platforms to provide:

- Real-time security alerts.
- Centralized log analysis.
- Improved event correlation.
- Historical reporting.

Such enhancements would simplify system administration and improve security visibility.

13.3.4 Expanded Validation

Additional testing could include:

- Large-scale email traffic.
- Multiple simultaneous users.

- Different attachment types.
- Additional malware simulation techniques.
- Long-duration stability testing.

These tests would provide a broader evaluation of system performance under varying operating conditions.

13.3.5 Production Deployment

The current implementation serves as a proof-of-concept developed within a virtual laboratory.

A future deployment could adapt the same architecture for a production environment by integrating public DNS services, trusted SSL/TLS certificates, backup strategies, and enterprise-level monitoring solutions.

13.4 Lessons for Future Projects

This project demonstrates that effective email security can be achieved through the integration of multiple open-source technologies.

Future implementations should continue to emphasize:

- Layered security.
- Careful planning.
- Incremental deployment.
- Continuous validation.
- Comprehensive documentation.

These practices improve both the reliability and maintainability of complex infrastructure projects.

13.5 Chapter Summary

This chapter discussed the limitations of the implemented Mail Security Gateway and presented several practical recommendations for future improvement. Although the project was developed within a controlled laboratory environment, it successfully demonstrated the implementation of a

layered email security solution using open-source technologies. Future enhancements such as secure email authentication, centralized monitoring, expanded validation, and production deployment could further strengthen the system while building upon the foundation established during this internship.

Chapter 14

Conclusion

14.1 Introduction

The successful implementation of a secure email infrastructure requires much more than installing a mail server. Modern email systems must be capable of protecting users against spam, malware, and unauthorized SMTP activity while maintaining reliable communication between clients and mail services.

This internship project focused on designing, implementing, and validating a Mail Security Gateway using open-source technologies within a controlled virtual laboratory. Throughout the implementation, multiple security components were integrated to create a layered email security architecture capable of inspecting incoming messages before delivery to the Internal Mail Server.

14.2 Project Summary

The project began with the design of a virtual laboratory environment consisting of four virtual machines that simulated a secure enterprise email infrastructure. A dedicated Internal Mail Server and a separate Mail Security Gateway were deployed to separate email delivery from security inspection.

The Internal Mail Server was configured using **Postfix** and **Dovecot** to provide reliable email delivery and mailbox access. A dedicated **GatewayVM** was then deployed to receive all incoming SMTP traffic before forwarding approved messages to the Internal Mail Server.

To strengthen the security of the gateway, several open-source technologies were integrated into the solution. **Rspamd** was configured to analyze incoming email, while **ClamAV** was integrated to detect malicious attachments. **UFW** was implemented to restrict unnecessary network access, and **Fail2Ban** was configured to monitor suspicious SMTP activity.

Following the deployment, the complete system was validated through a series of controlled test scenarios that demonstrated legitimate email delivery, malware detection using the EICAR antivirus test file, and the system's behaviour during increased SMTP connection activity.

14.3 Achievement of Objectives

The implementation successfully achieved the objectives established at the beginning of the internship.

The completed system demonstrated the ability to:

- Deploy a functional Internal Mail Server.
- Implement a dedicated Mail Security Gateway.
- Relay legitimate email through the gateway.
- Inspect incoming email before delivery.
- Detect malicious attachments using ClamAV.
- Analyze messages using Rspamd.
- Strengthen gateway security using UFW and Fail2Ban.
- Validate the implemented security architecture through practical testing.

The successful completion of these objectives confirms that the Mail Security Gateway operated as intended within the scope of the internship.

14.4 Technical Contributions

The project demonstrates how multiple open-source technologies can be integrated to provide layered email protection.

Rather than depending on a single security mechanism, the implemented solution distributes security responsibilities across several independent components. This layered approach improves the overall reliability of the system while reducing the likelihood of malicious email reaching the Internal Mail Server.

The project also demonstrates the importance of systematic planning, incremental deployment, configuration verification, and practical validation when implementing complex Linux-based infrastructure.

14.5 Practical Experience Gained

One of the most valuable outcomes of this internship was the opportunity to apply theoretical knowledge in a practical environment.

Throughout the implementation, practical experience was gained in several important technical areas, including:

- Linux server administration.
- Virtualization using Oracle VirtualBox.
- SMTP communication.
- Mail server deployment.
- Email security implementation.
- Spam filtering.
- Malware detection.
- Firewall configuration.
- Service integration.
- System troubleshooting.
- Log analysis.
- Technical documentation.

These experiences significantly improved both technical knowledge and problem-solving skills while providing a better understanding of real-world infrastructure deployment.

14.6 Final Remarks

The implementation of the Mail Security Gateway successfully demonstrated that open-source technologies can be integrated to create a secure and reliable email infrastructure.

Although the project was developed within a virtual laboratory environment, it reflects many of the design principles used in modern enterprise email security solutions, including layered defense, service separation, and proactive email inspection.

The knowledge and practical experience gained throughout this internship provide a strong foundation for future work in Linux system administration, cybersecurity, and secure infrastructure management.

Overall, this internship not only resulted in the successful completion of the assigned project but also provided valuable hands-on experience that bridges the gap between academic learning and professional practice.

14.7 Chapter Summary

This chapter concluded the implementation of the Mail Security Gateway by summarizing the project objectives, implementation process, validation activities, and technical outcomes. The project successfully demonstrated the deployment of a layered email security solution using open-source technologies while providing valuable practical experience in Linux administration, email security, and infrastructure deployment. The following and final chapter reflects on the overall internship experience and the professional skills developed throughout the training period.